

Image Classification

CVAI



20. März 2025

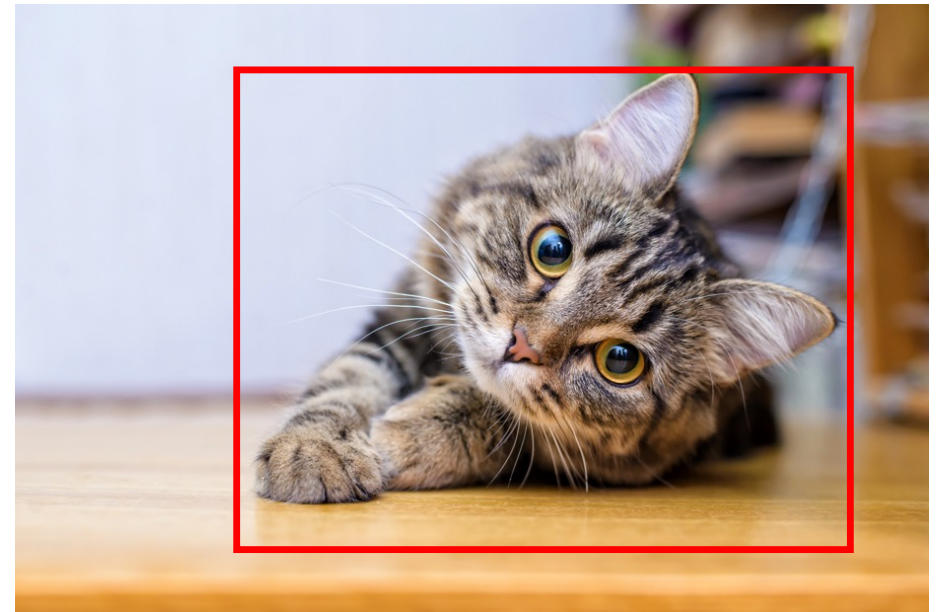
FH Zentralschweiz

Problems in Computer Vision

Image Classification

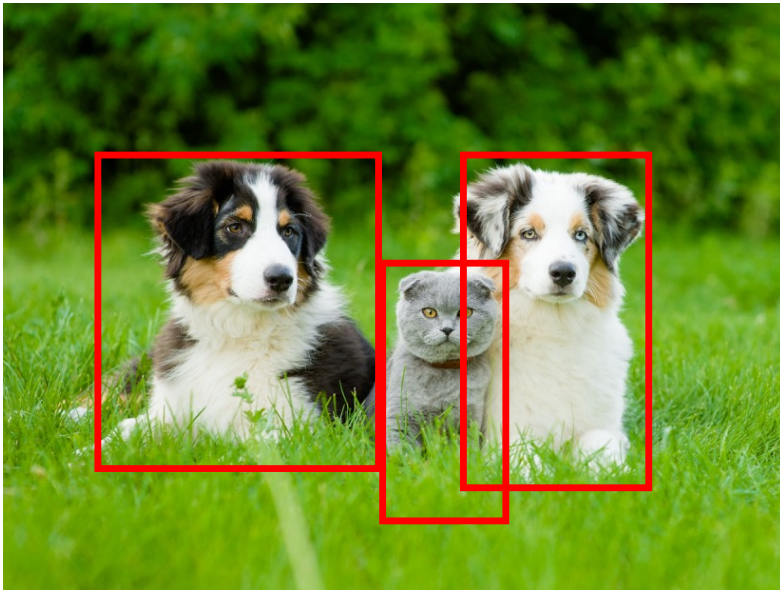


Classification and Localization



Problems in Computer Vision

Object Detection and Classification



Semantic Segmentation



Problems in Computer Vision

Image Classification

Image $\mapsto$ Class



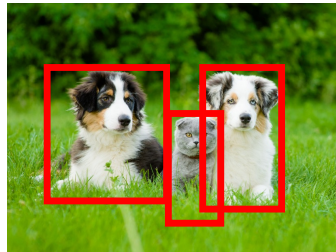
Today and next week

Examples:

- ...

Object Detection

Image $\mapsto$ List[(Region, Class)]



next week (SW6)

Examples:

- ...

Semantic Segmentation

Image $\mapsto$ Array[Class]



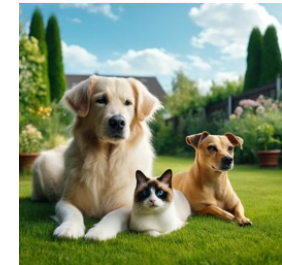
SW7

Examples:

- ...

Image Generation

Text $\mapsto$ Image



SW8

Examples:

- ...

Introduction to Image Classification / Object Recognition

$f(\text{cat image}) = \text{cat}$

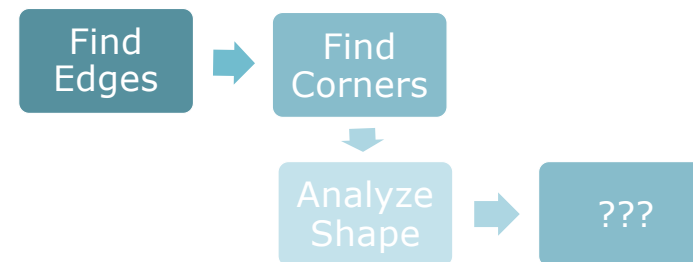
$f(\text{dog image}) = \text{dog}$

We would like to have a function where we can give the image as input, and it will return the class of the image.

It is not obvious how to program an algorithm for recognizing different object classes!

This has been tried bottom-up

```
def classify(image) -> int:  
    # what to do here  
  
    return class_label
```



Object recognition: Is it really so hard?

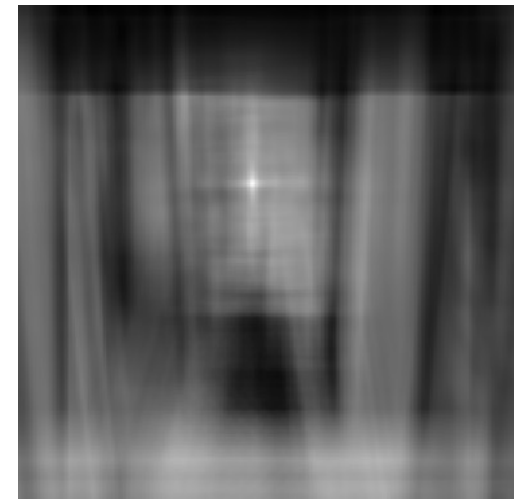
This is a chair



Find the chair in this image



Output of normalized correlation

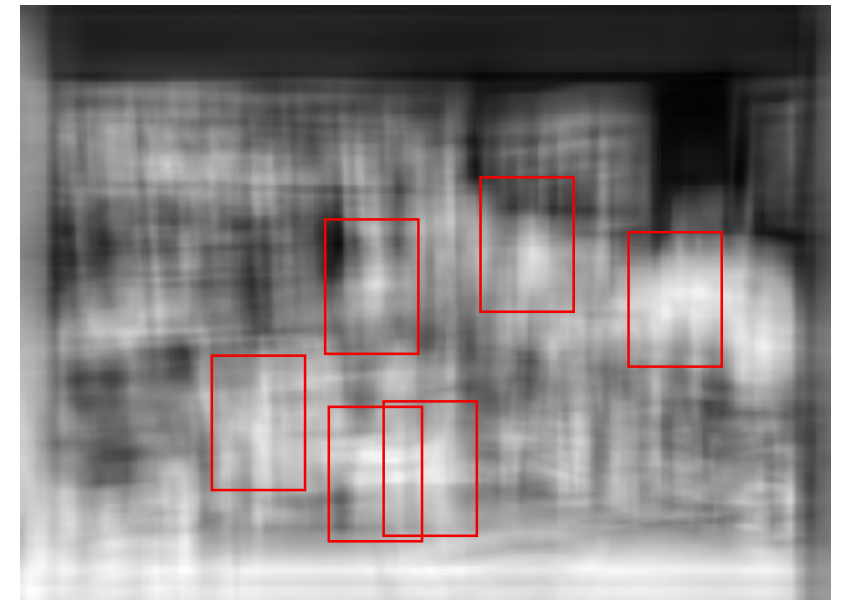
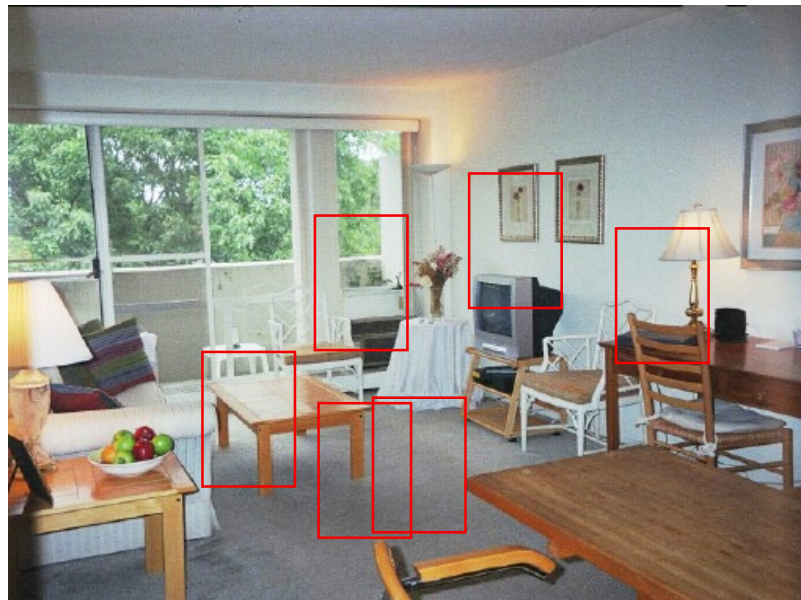


Object recognition: Is it really so hard?



Pretty much unusable

Object recognition: Is it really so hard?



A “popular method is that of template matching, by point to point correlation of a model pattern with the image pattern. These techniques are inadequate for three-dimensional scene analysis for many reasons, such as occlusion, changes in viewing angle, and articulation of parts.” Nivatia & Binford, 1977.

Object Recognition approaches

Before Machine Learning:

- Contour Extraction (Edge Detection and Grouping), Analysis of the contours by shape descriptors
- Template Matching

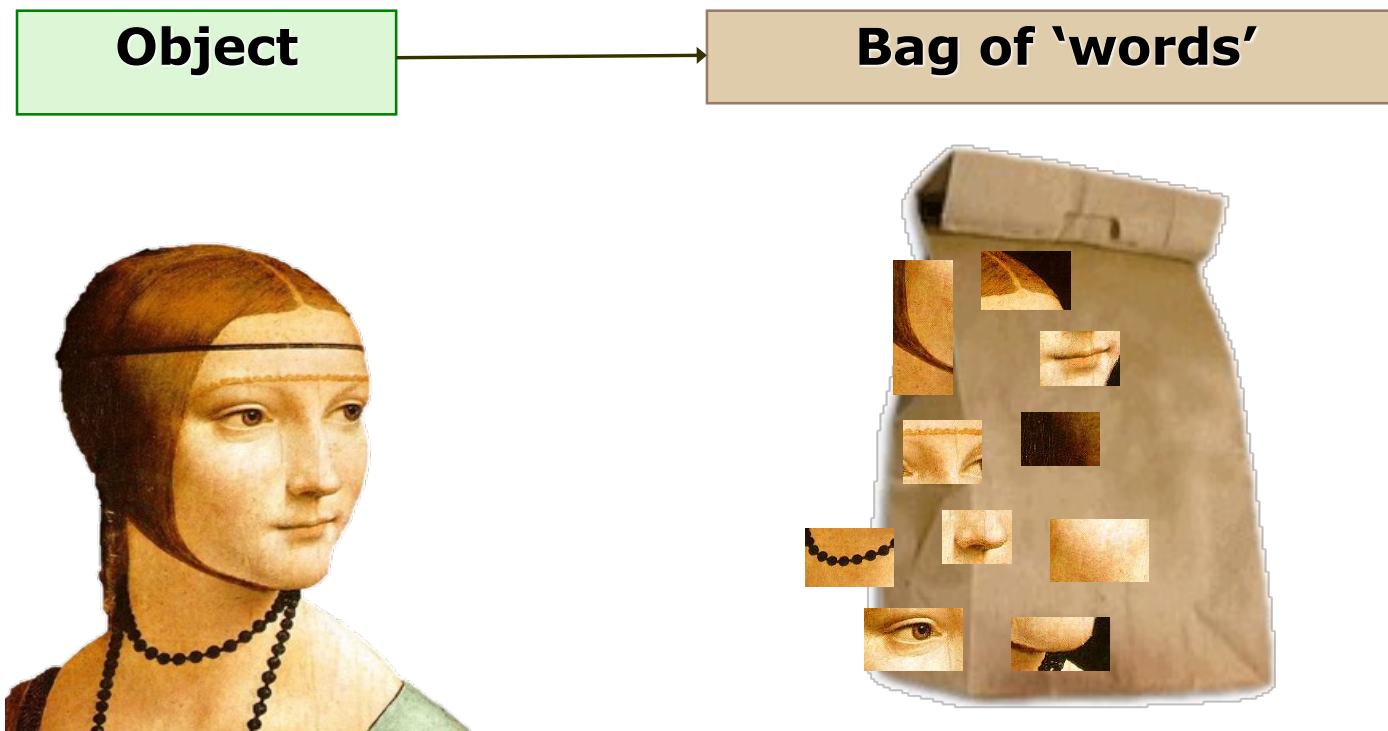
Features and Machine Learning (not neural networks):

- SIFT, SURF, ORB, HOG and other features combined with machine learning (SVM)
- Bag of Word approaches (code words) and classification

Deep neural networks:

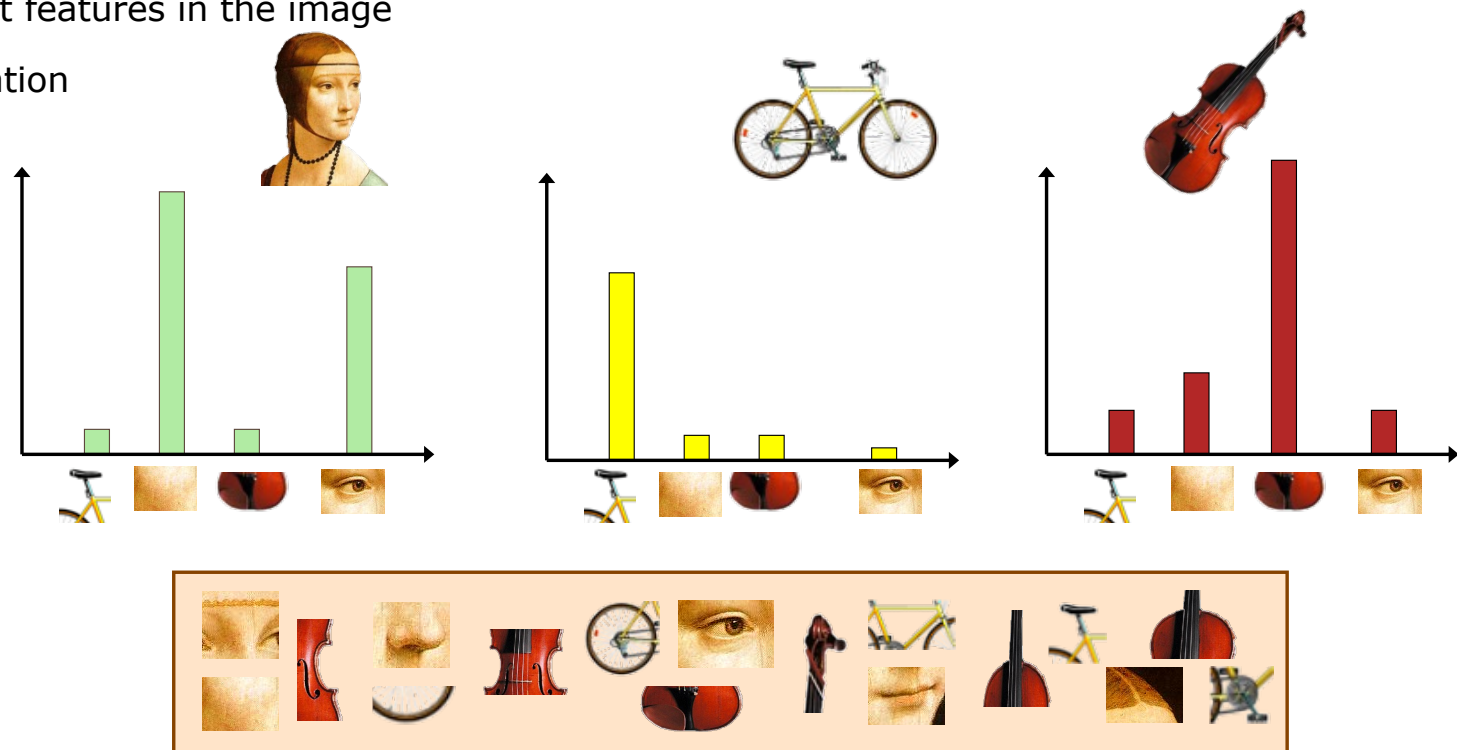
- Use Deep Learning for feature extraction and classification

Approaches before Neural Networks: Bag of Words Model

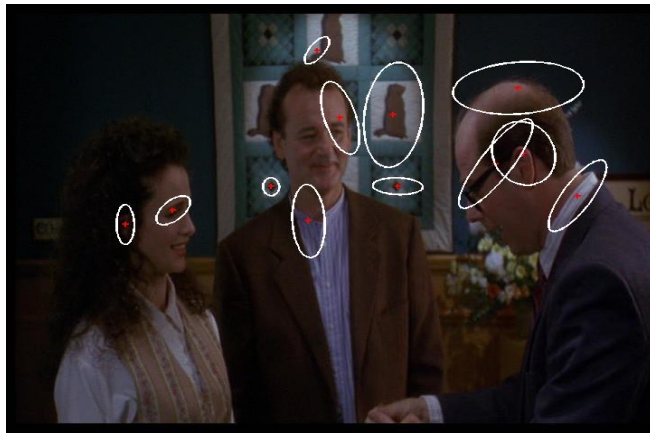


Bag of Words

- Calculate independent features in the image
- Histogram representation
- "Classify" histograms

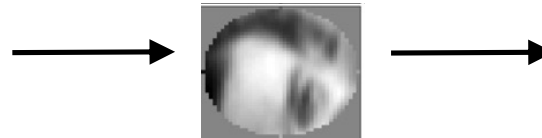


Feature Detection and Representation

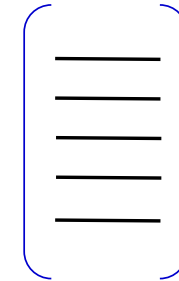


Detect patches

Local interest operator or on
regular grid

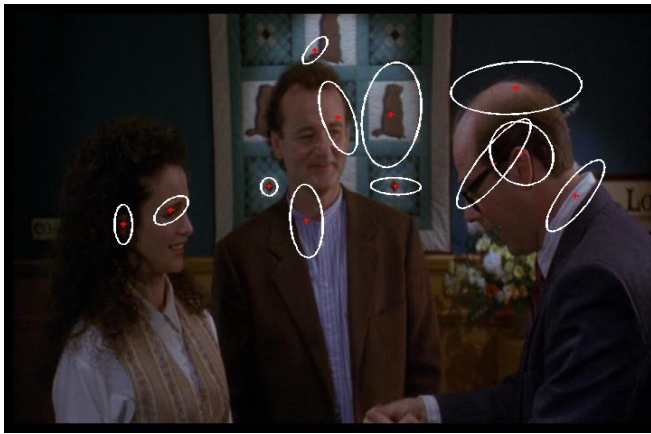


Normalize patch



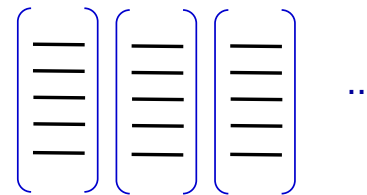
Compute Descriptor

Feature Detection and representation



Detect patches

Local interest operator or
regular grid



Normalize patch

Compute Descriptor

Codewords dictionary formulation

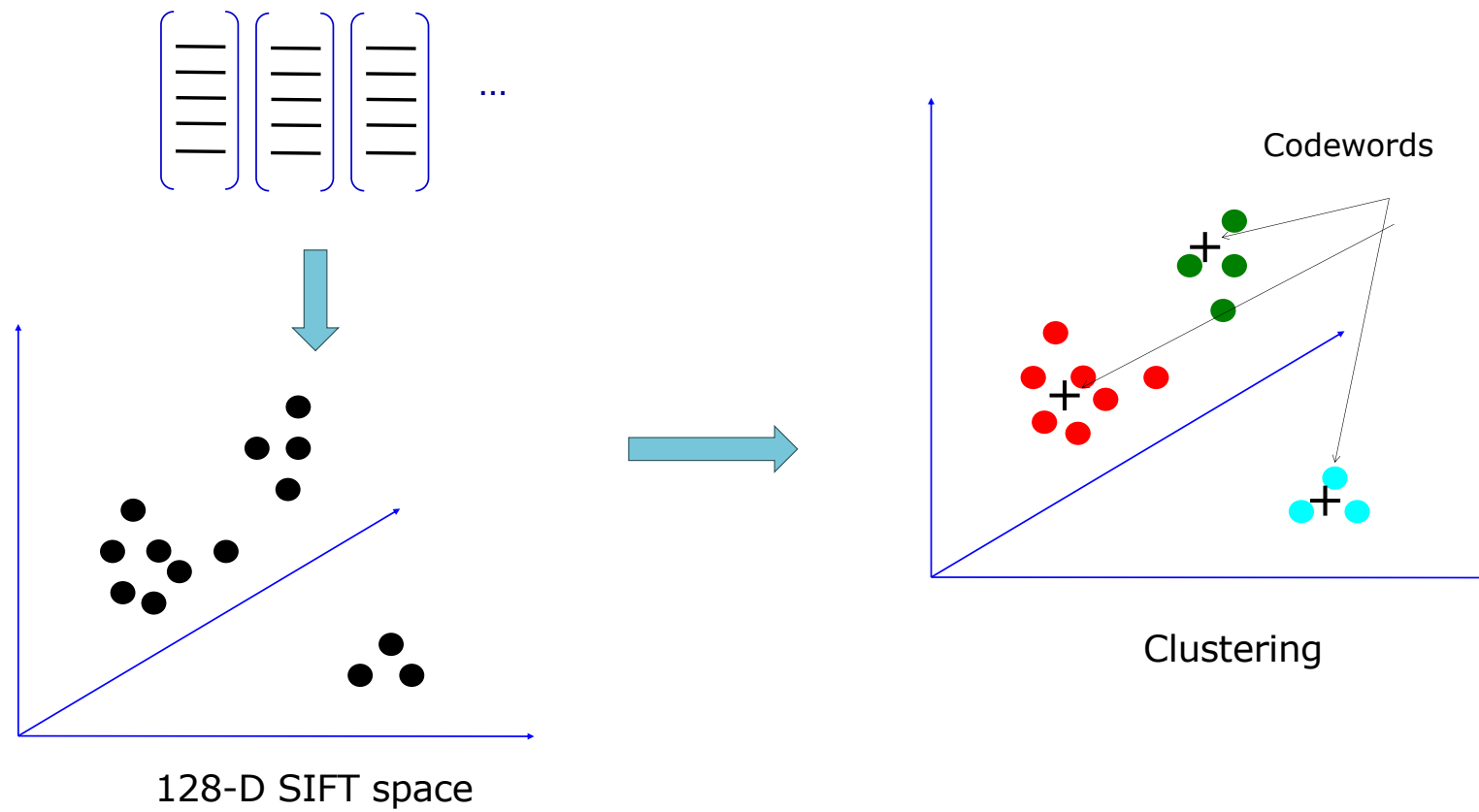


Image Patch Examples

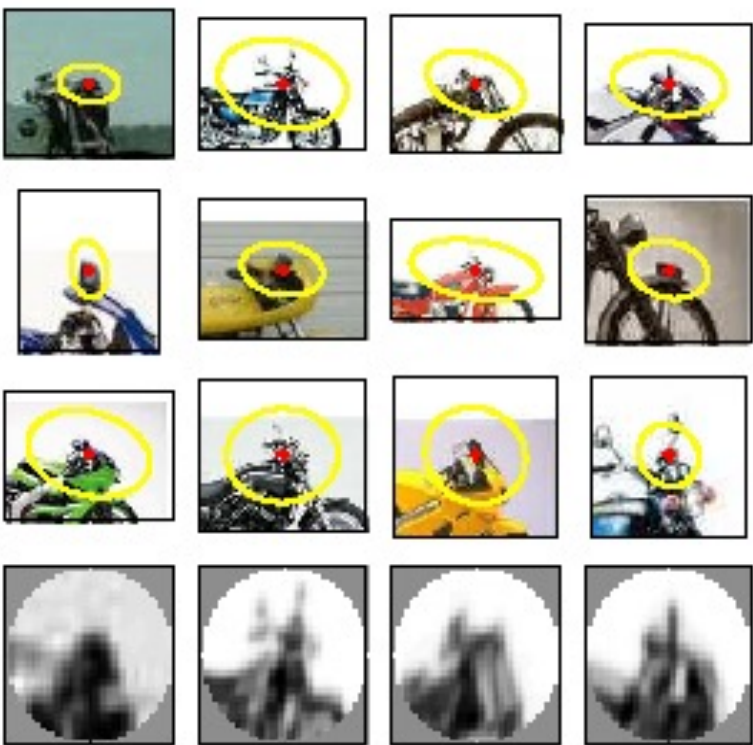
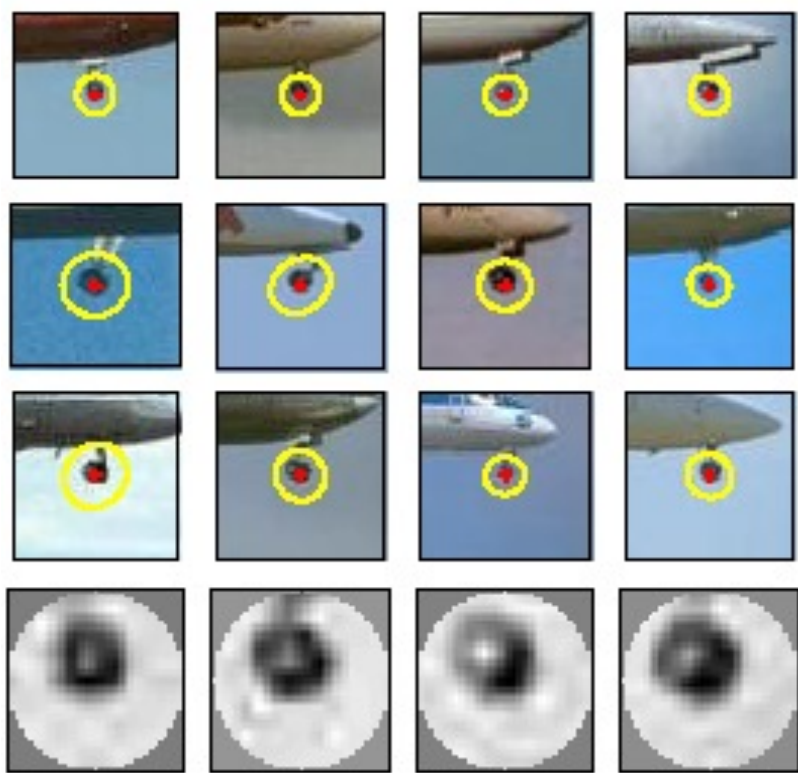
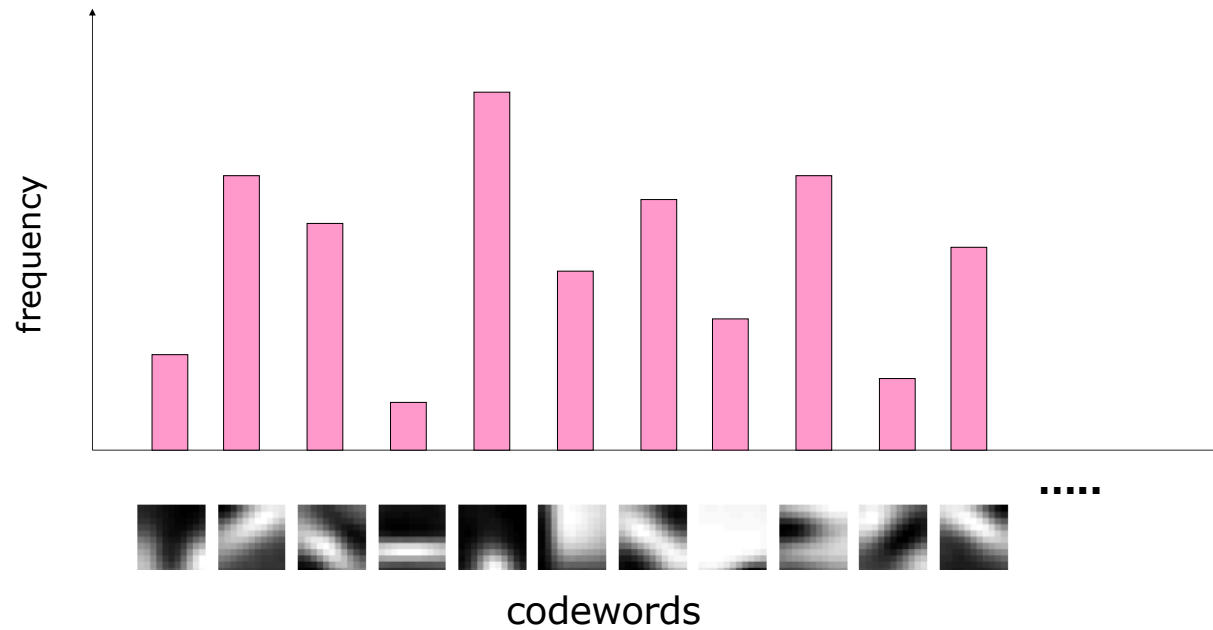


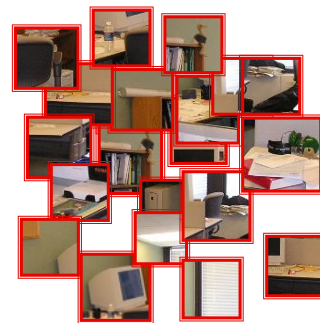
Image Representation

Histogram of features
assigned to each cluster



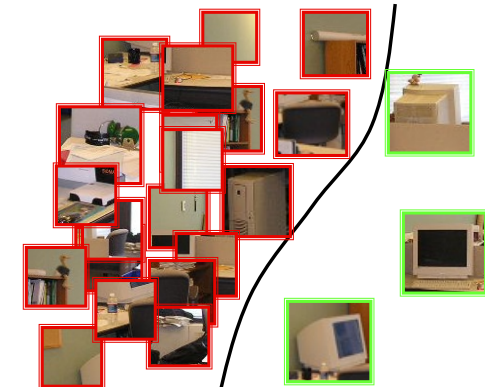
Classifier based methods

- Object detection and recognition is formulated as a classification problem.
- The image is partitioned into a set of overlapping windows
- ... and a decision is taken at each window about if it contains a target object or not.



Bag of image patches

Background



Computer screen

In some feature space

Classification Repetition

"Classical data" set from Fisher 1936: Iris flowers



Iris setosa



Iris virginica



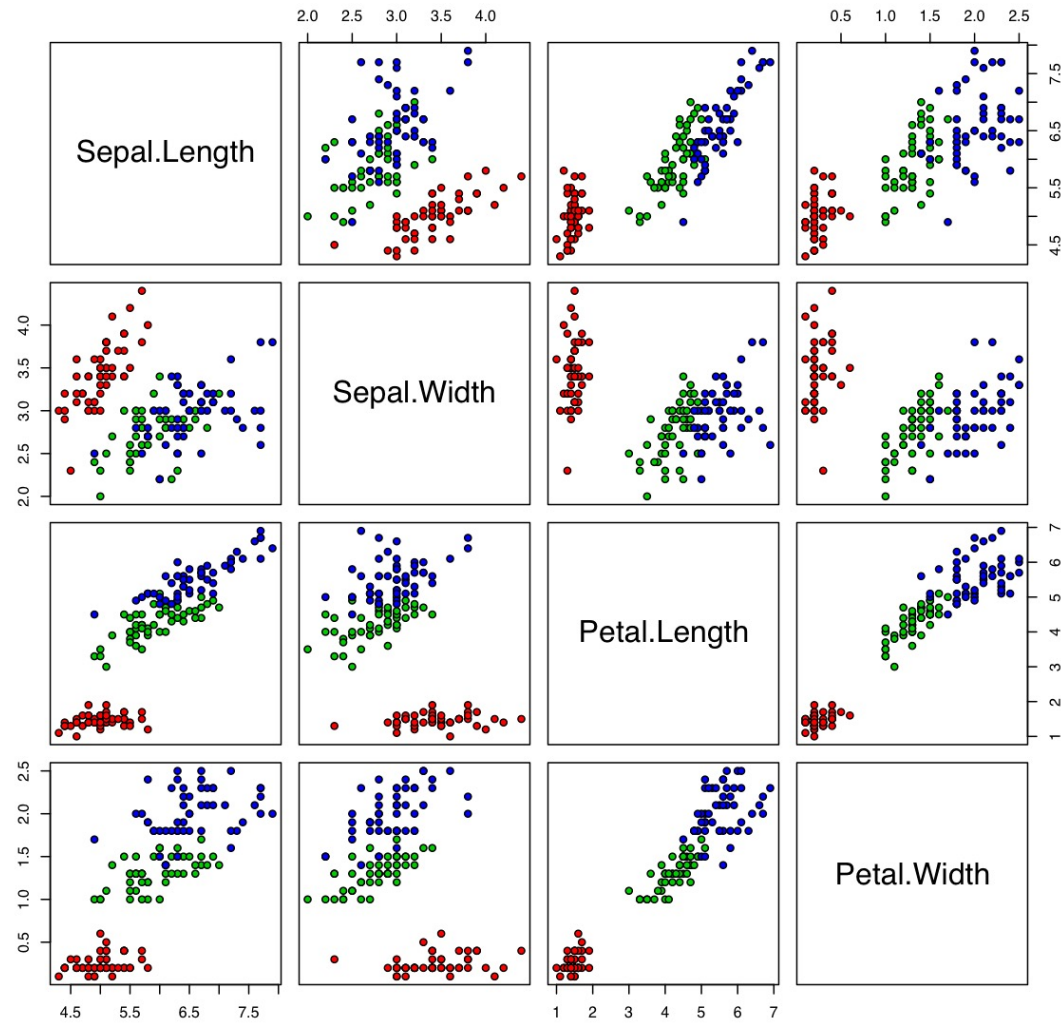
Petal
Kronblatt

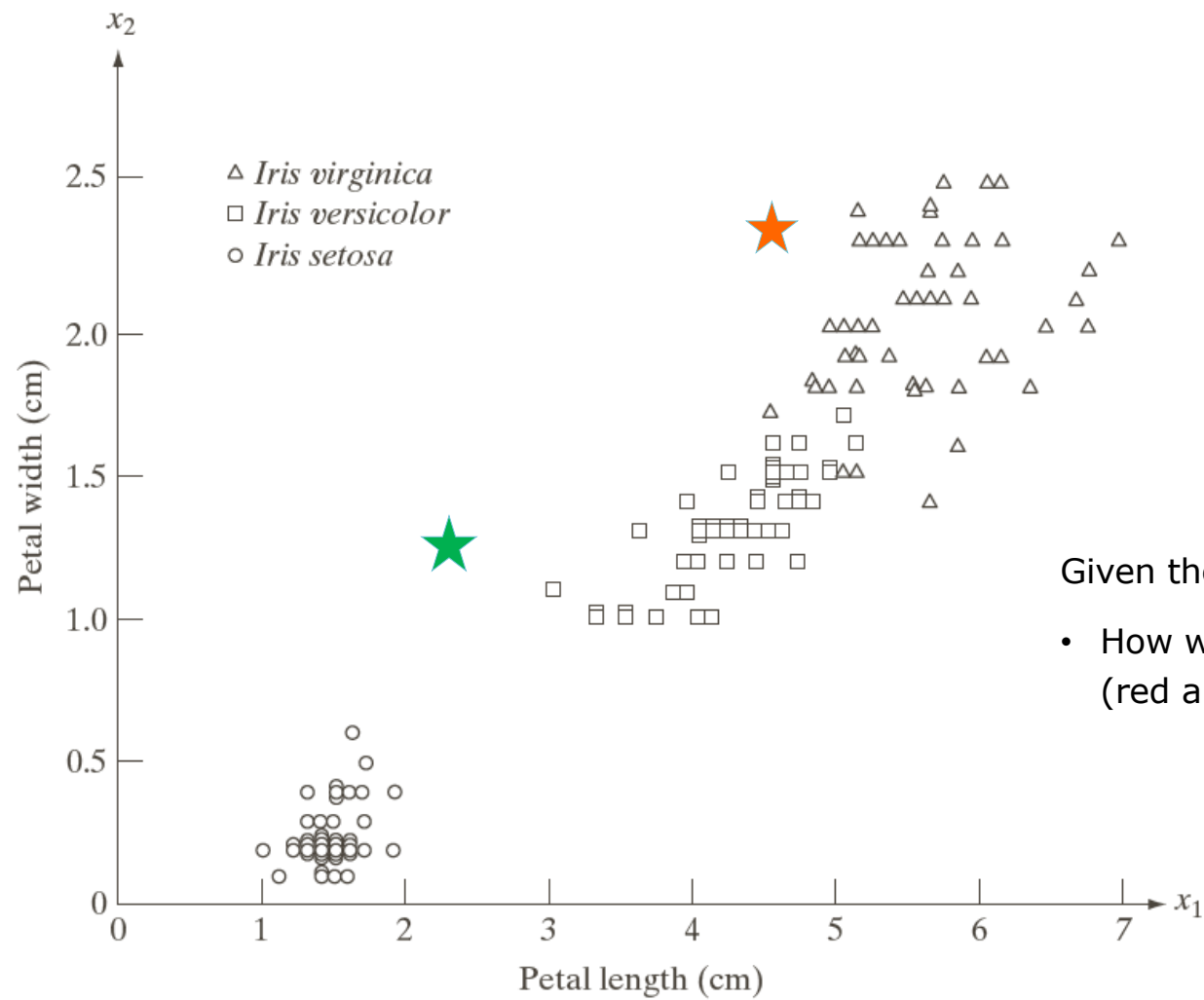
Sepal
Kelchblatt

Iris versicolor

https://en.wikipedia.org/wiki/Iris_flower_data_set

Iris Data (red=setosa,green=versicolor,blue=virginica)



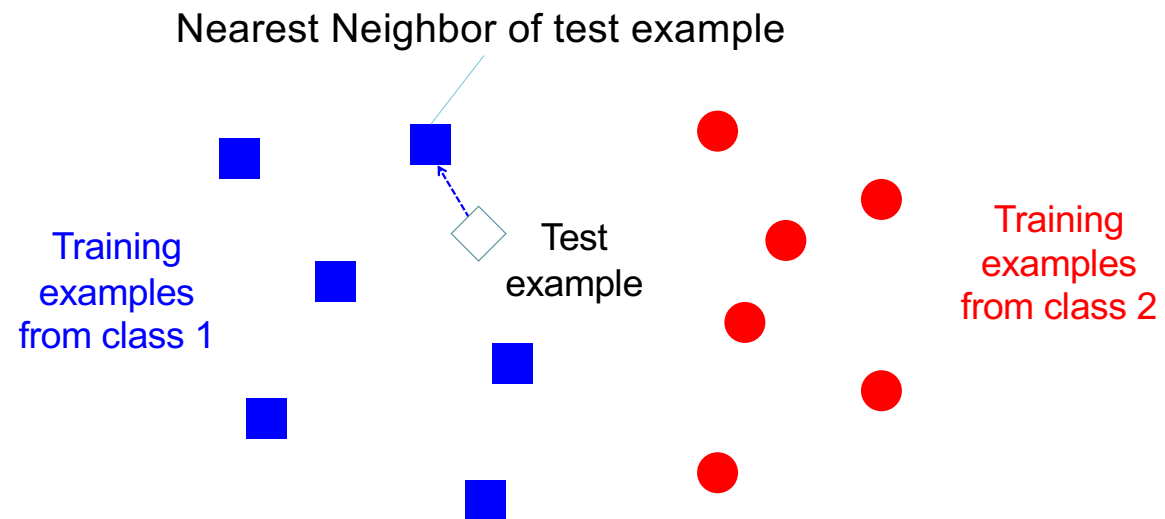


Given the existing data:

- How would you classify the new samples (red and green) ?

Classification: Nearest Neighbor

- Classify example according the label of the nearest training example
- Only a distance function is needed



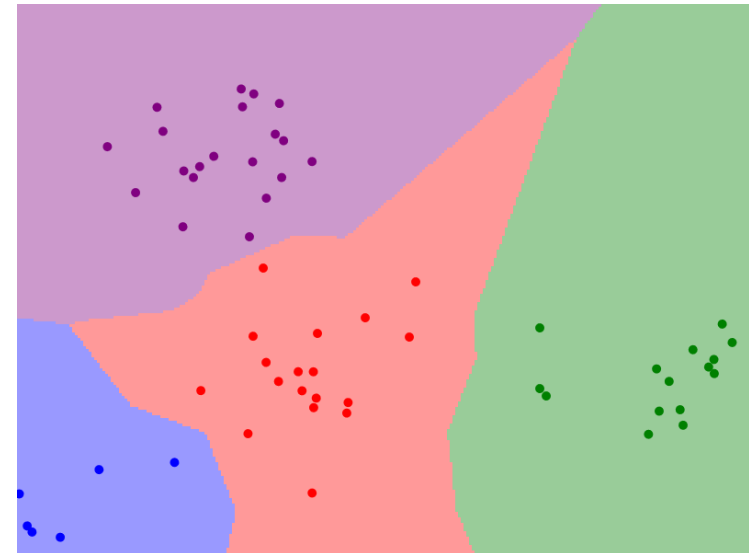
k Nearest Neighbor

Use the labels of the k nearest neighbors instead

Demo: <http://vision.stanford.edu/teaching/cs231n-demos/knn/>

Questions:

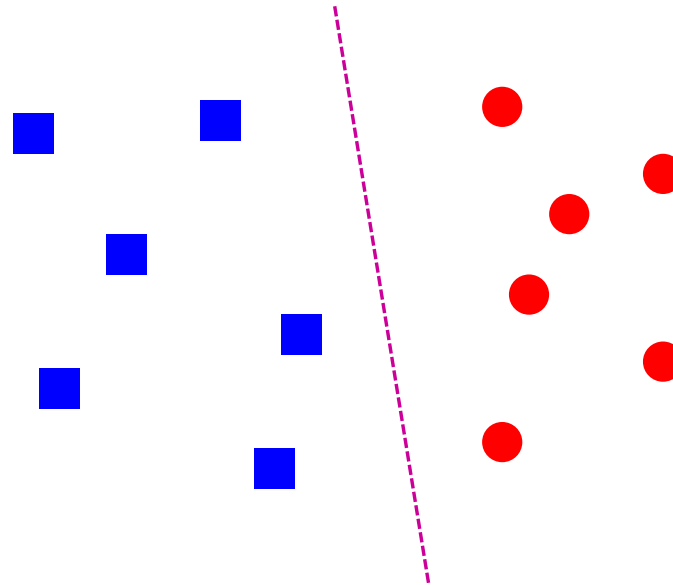
- Why are there more white areas when k is even?
- Can there be a label for class 1 in an area that is classified as class 2?
- How does the border between classes look like for $k=1$?
- What are the different distance functions?



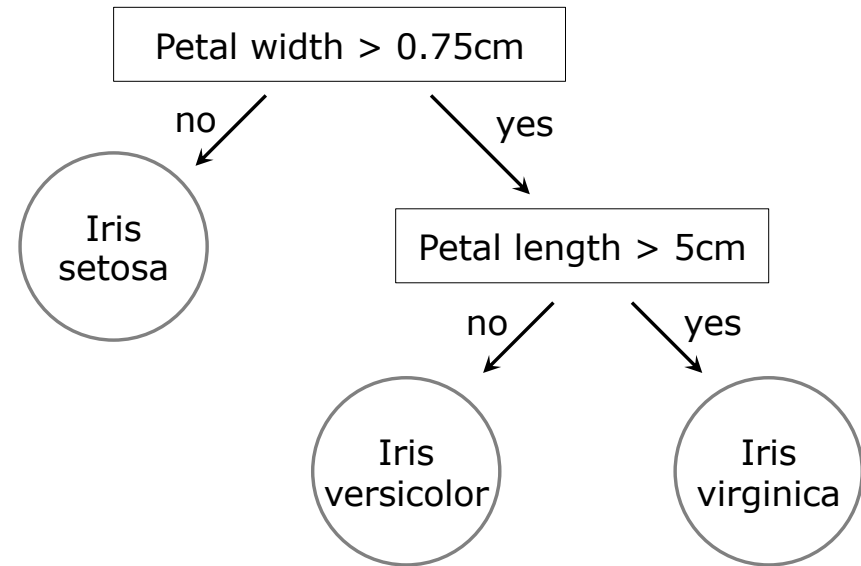
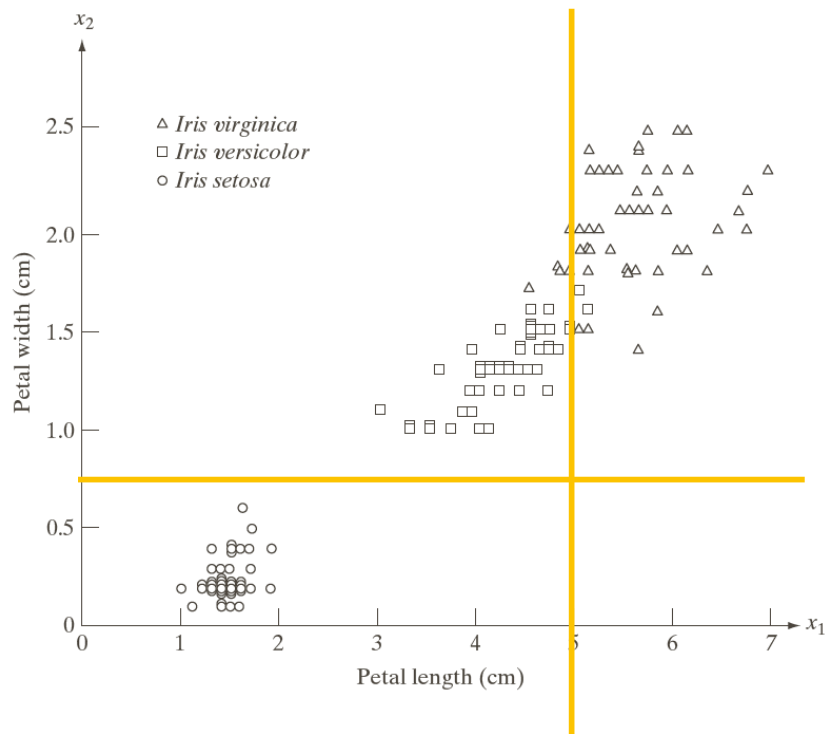
Classification: Linear function

Use a linear function (also called hyperplane) to distinguish between 2 classes

$$f(x) = wx + b$$



Decision Tree Classifier



Idea: Iteratively divide feature space, so that impurity of class labels (e.g., Gini impurity) decreases

Example: Face Detection





Nikon Advertisement

Viola Jones Face/Object-Detektor / Cascaded classifier

Problem:

- Sliding Window: Evaluate many position in the image if they contain a face
- How to scan many positions as efficiently as possible for possible faces?

Ideas:

- **Boosting** using Adaboost
- **Simple** Features
- **Cascaded Classifiers**

Robust Real-time Object Detection

Paul Viola
viola@merl.com
Mitsubishi Electric Research Labs
201 Broadway, 8th FL
Cambridge, MA 02139

Michael Jones
mjones@crl.dec.com
Compaq CRL
One Cambridge Center
Cambridge, MA 02142

Machine Learning Approach: Boosting

Idea:

Input: Training data (positive, negative) with **weights**

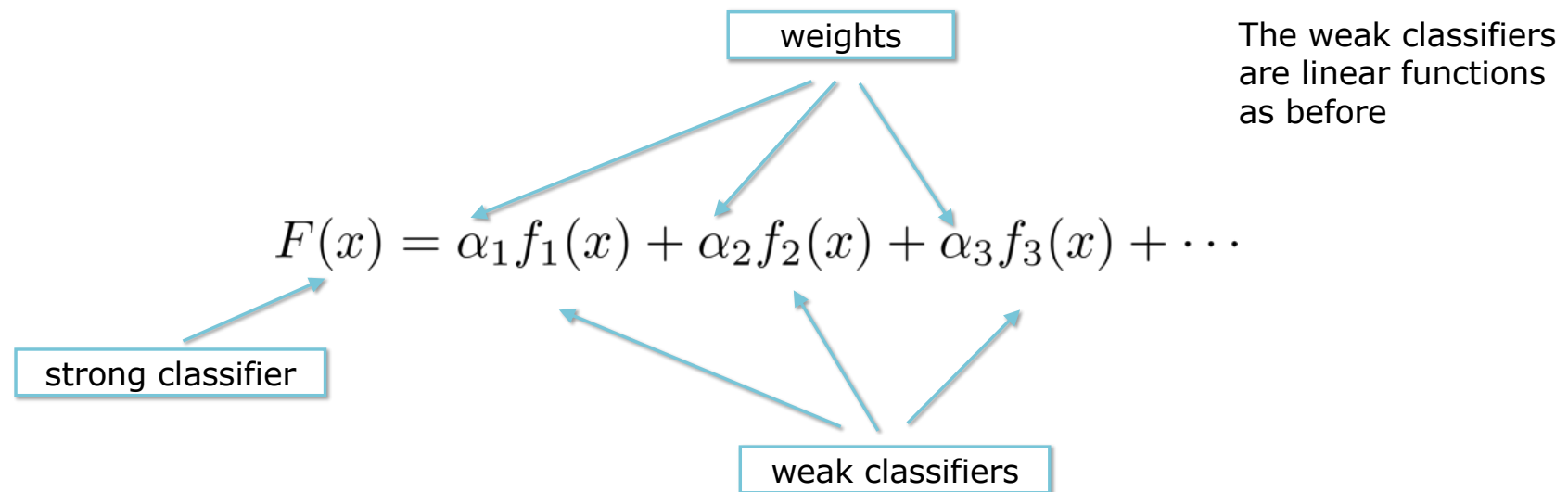
FOR $t=1 \dots T$

- Train a *weak* classifier
- Classify training data
- Increase weight on incorrectly classified samples

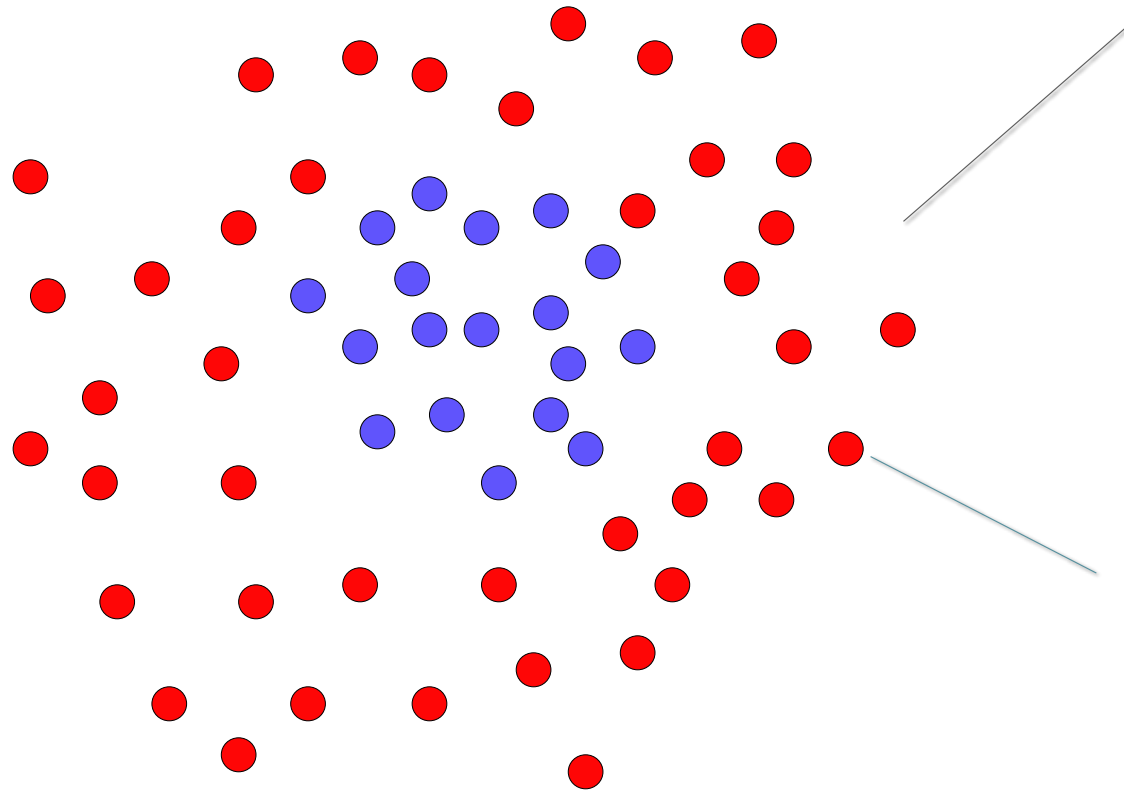
Use the weighted combination of all weak classifiers for the final (strong) classifier

Boosting

- *Strong* classifier from a linear combination of *weak* classifiers



Boosting

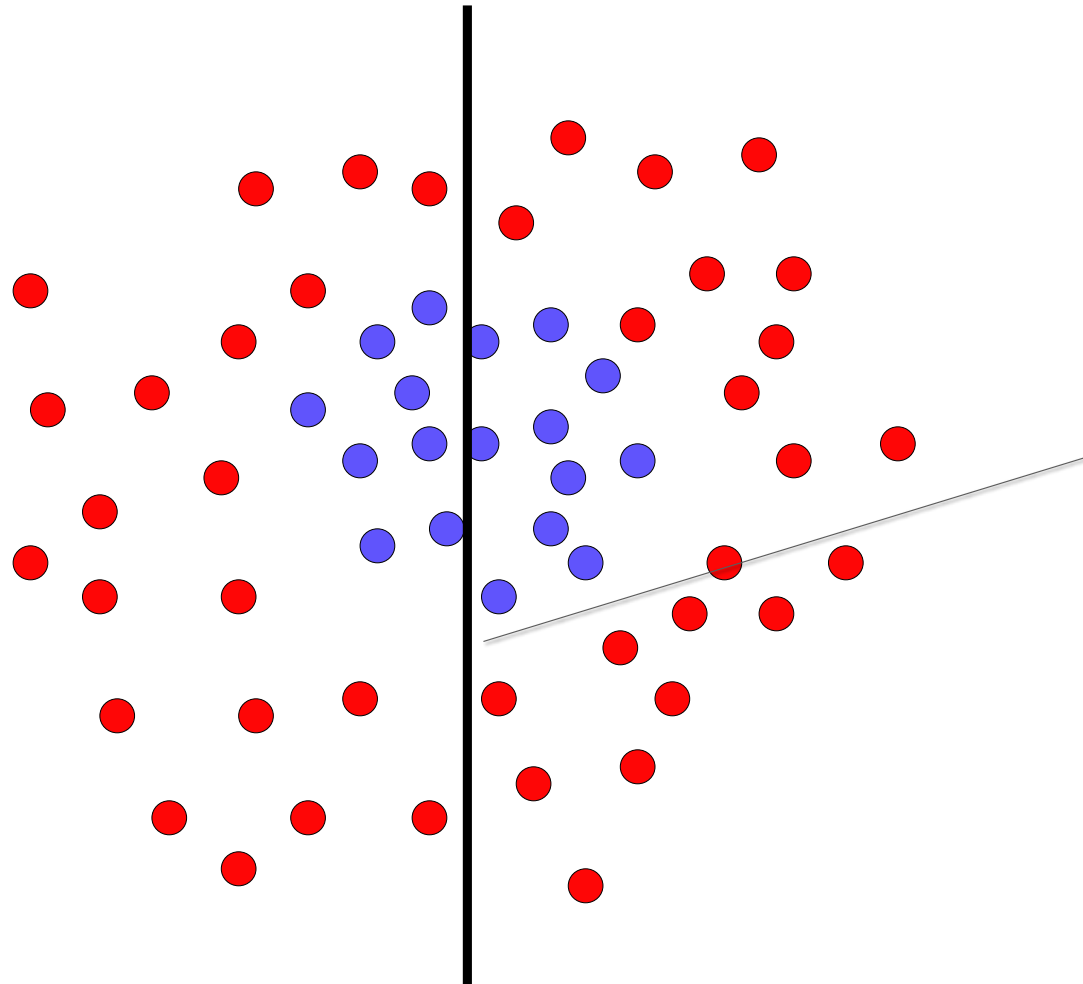


Goal:

Find a classifier that separates the red from the blue samples

Each data point has a class label (+1, -1) and a weight (initialized to 1)

Boosting

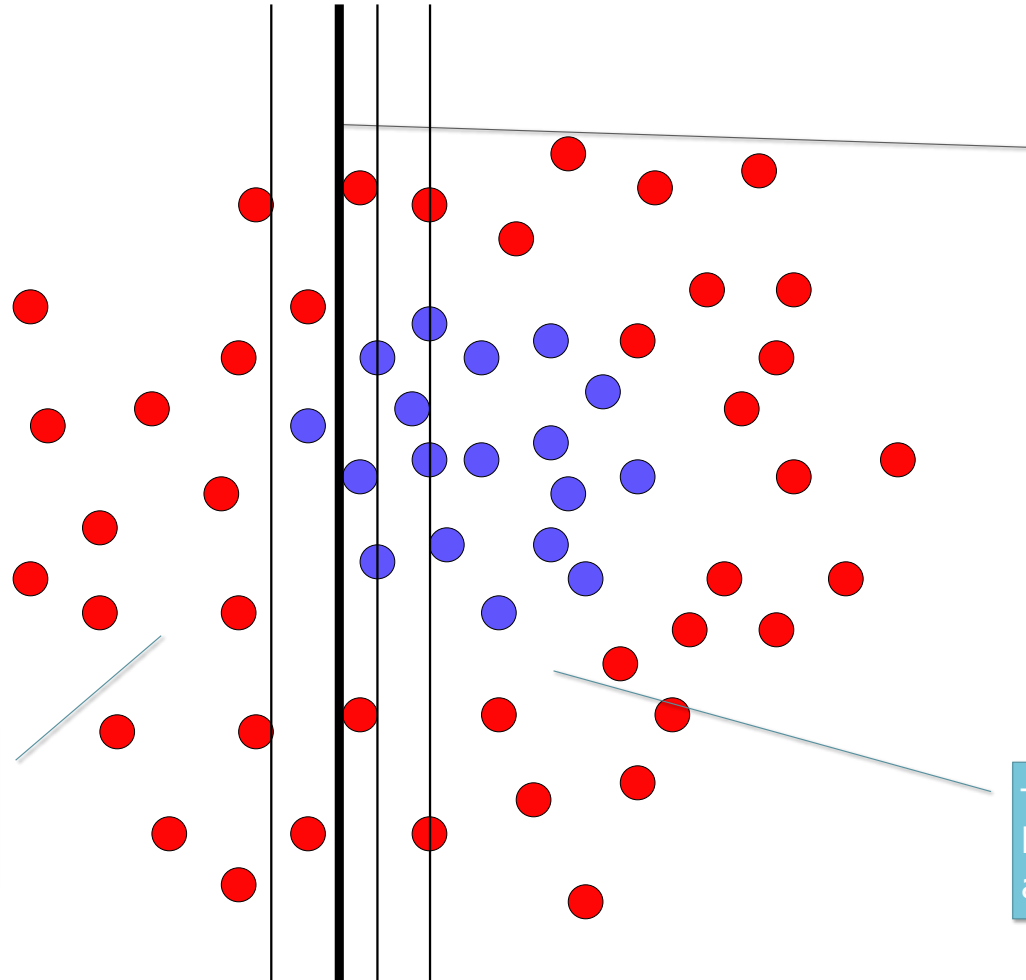


weak linear classifier

$$p(\text{error}) = 0.5$$

Boosting

This side will
be classified
as red

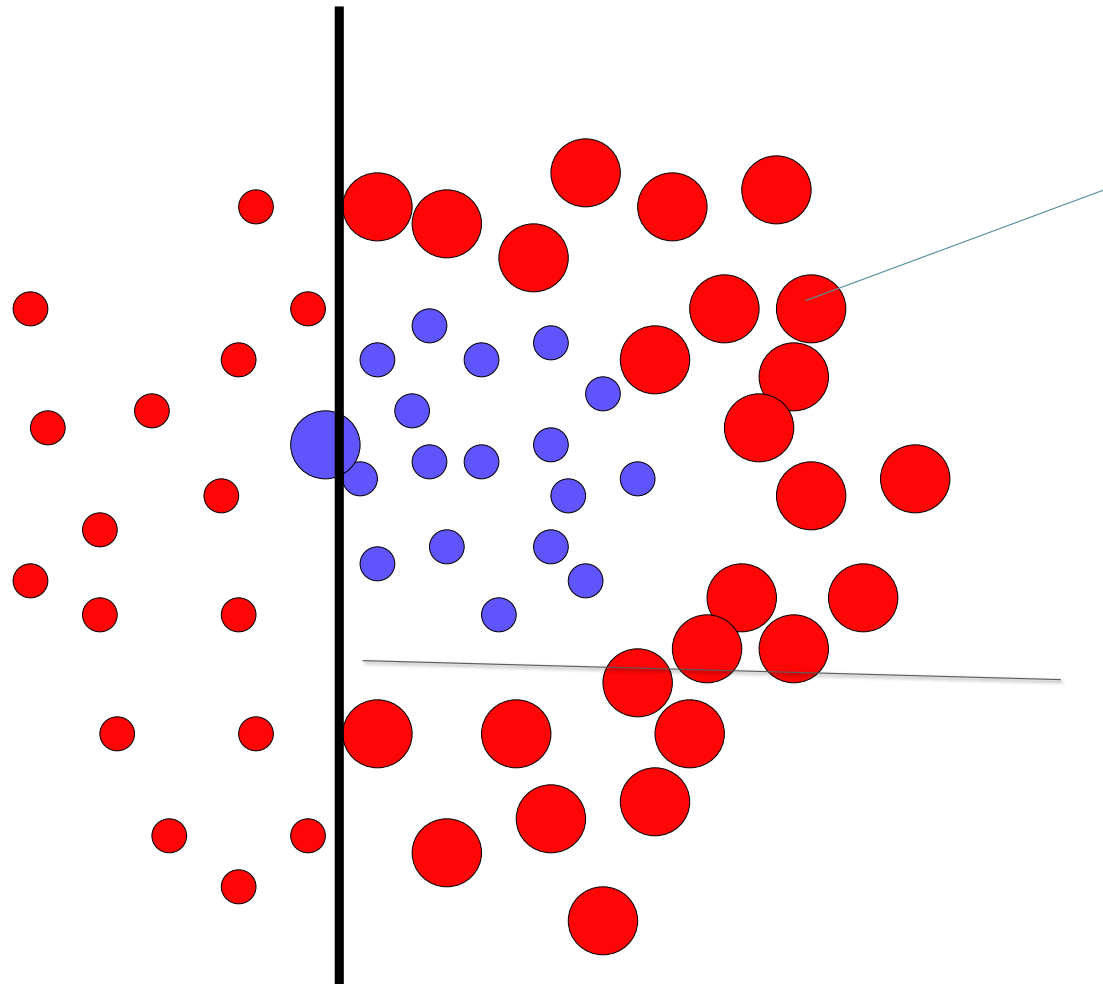


This one seems best

It is still a **weak classifier**: it performs slightly better than chance

This side will
be classified
as blue

Boosting

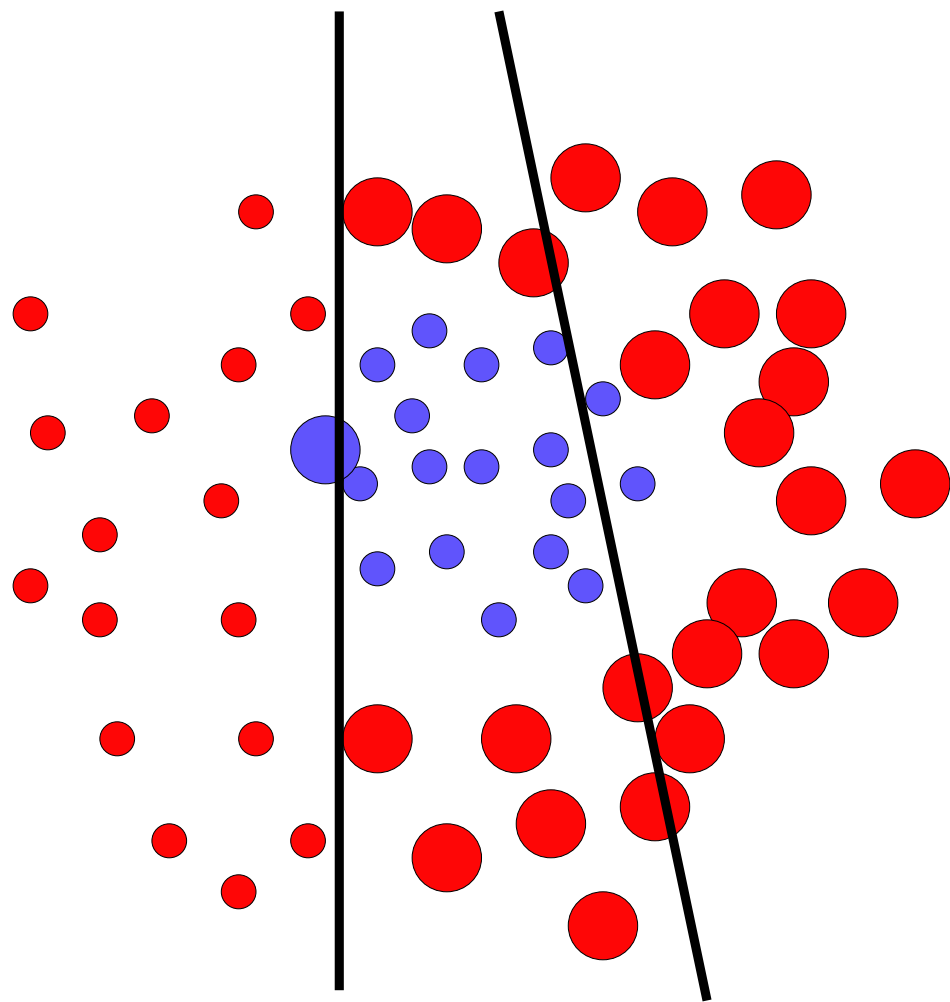


We update the weights:

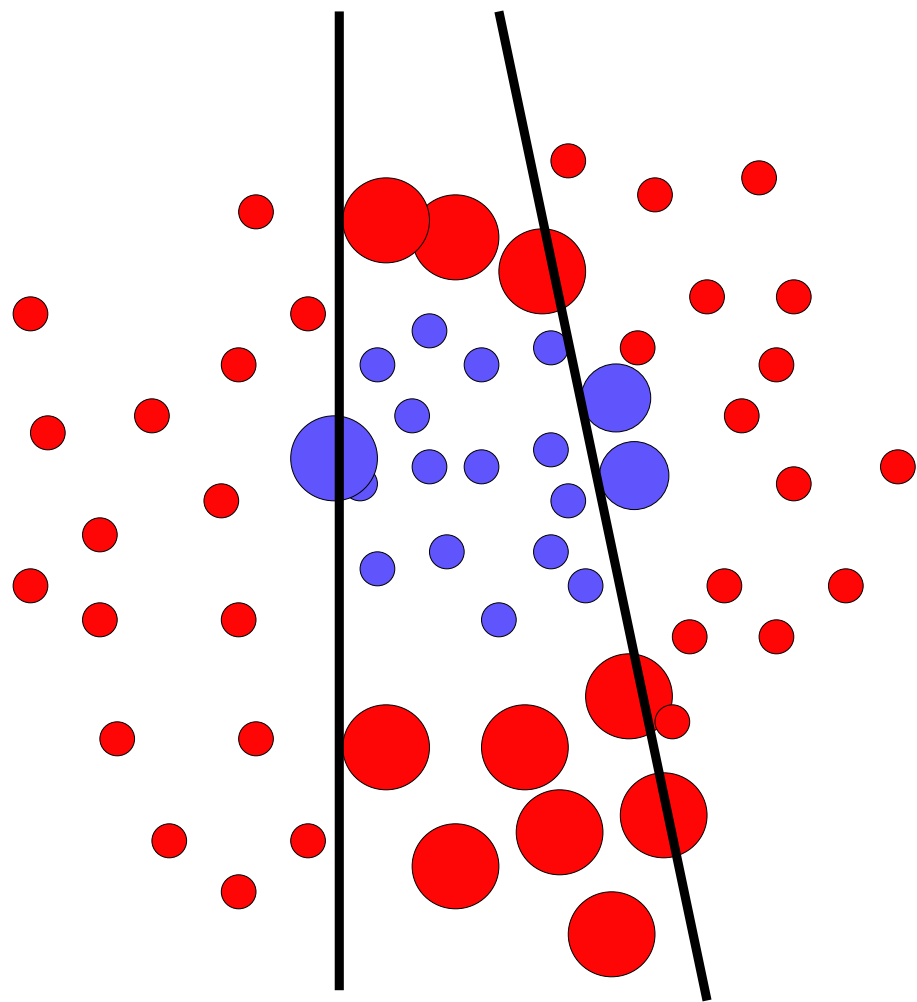
- Increase the weights for wrongly classified samples
- Decrease the weights for correctly classified samples

This sets a new problem for which the previous weak classifier performs at chance again

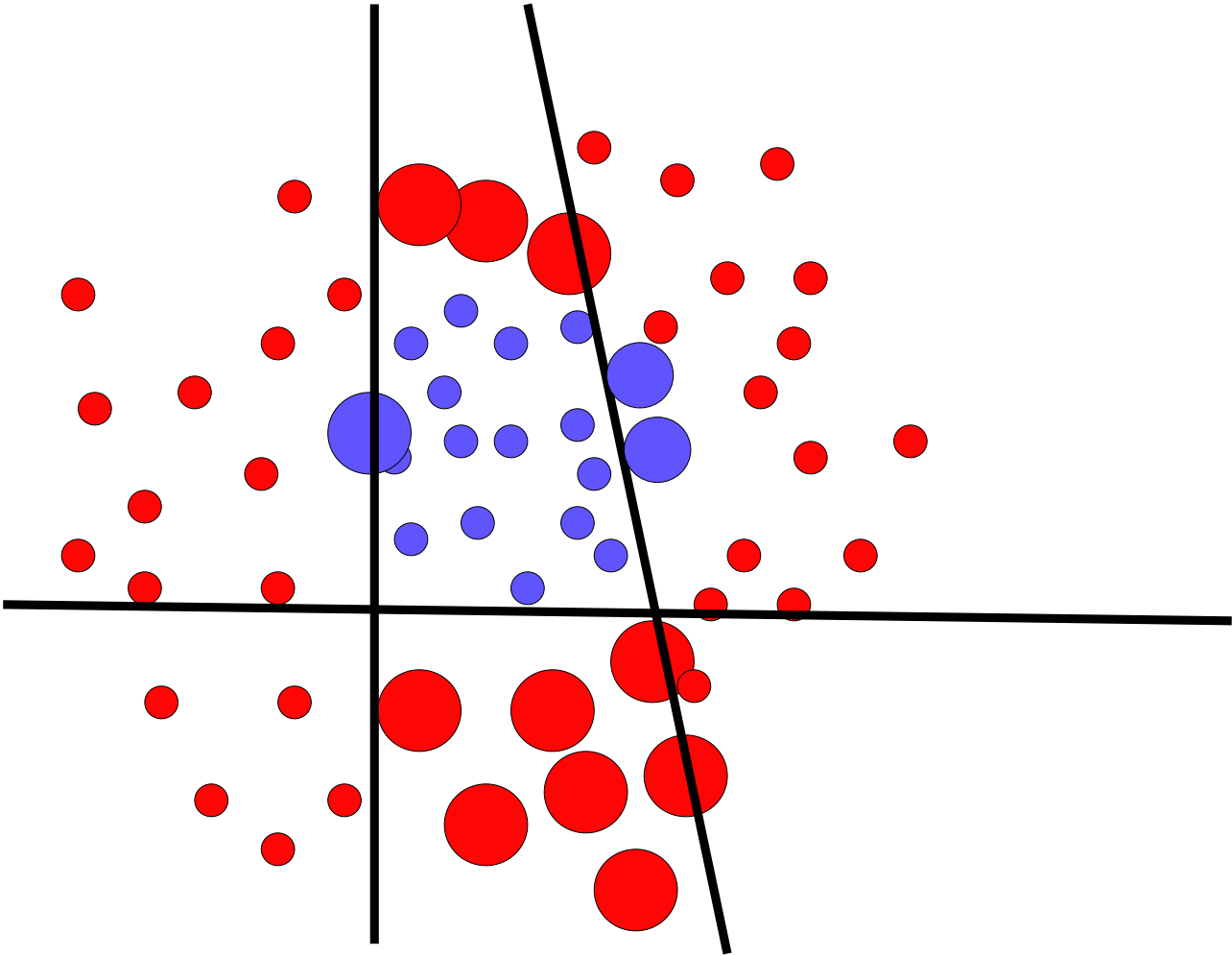
Boosting



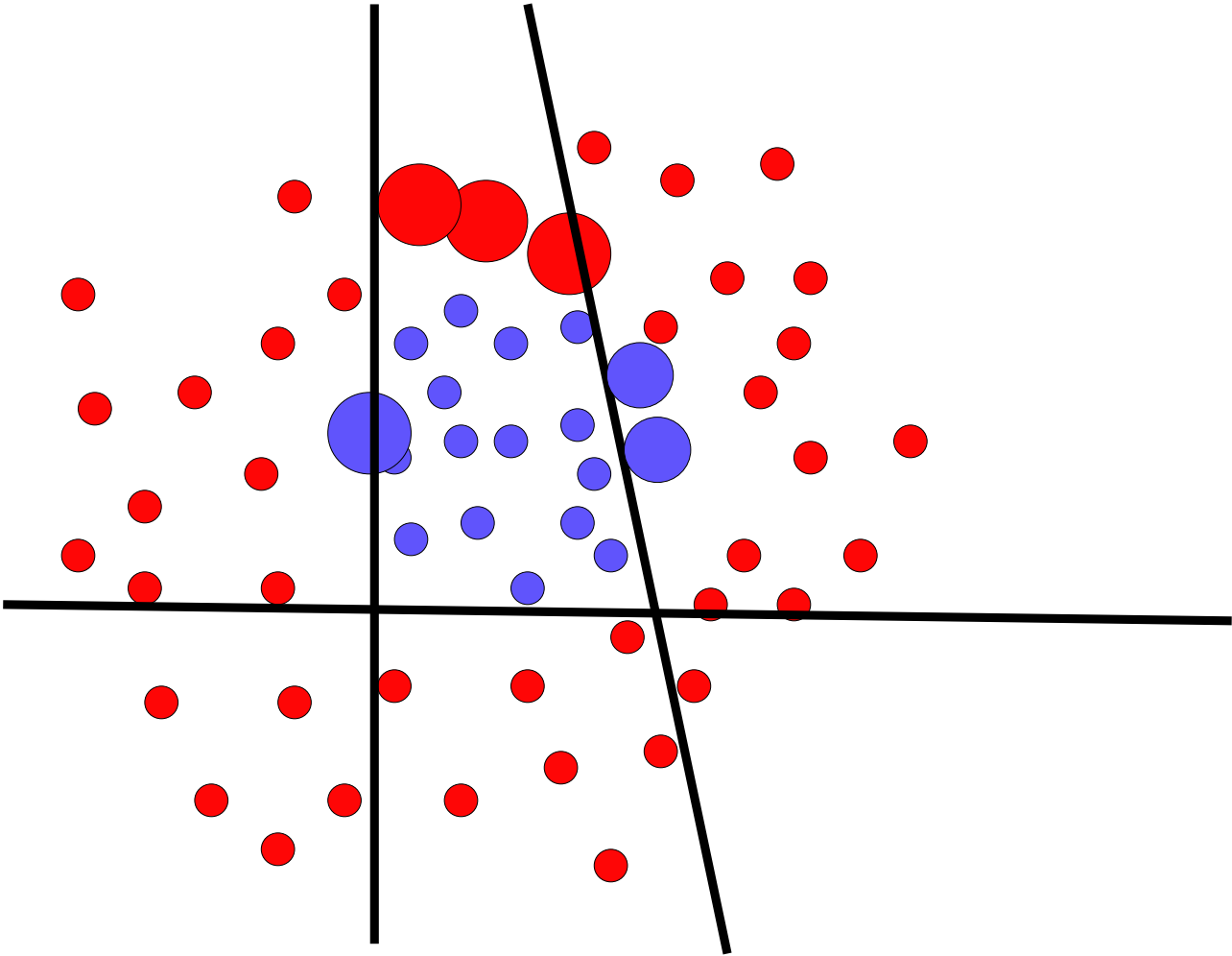
Boosting



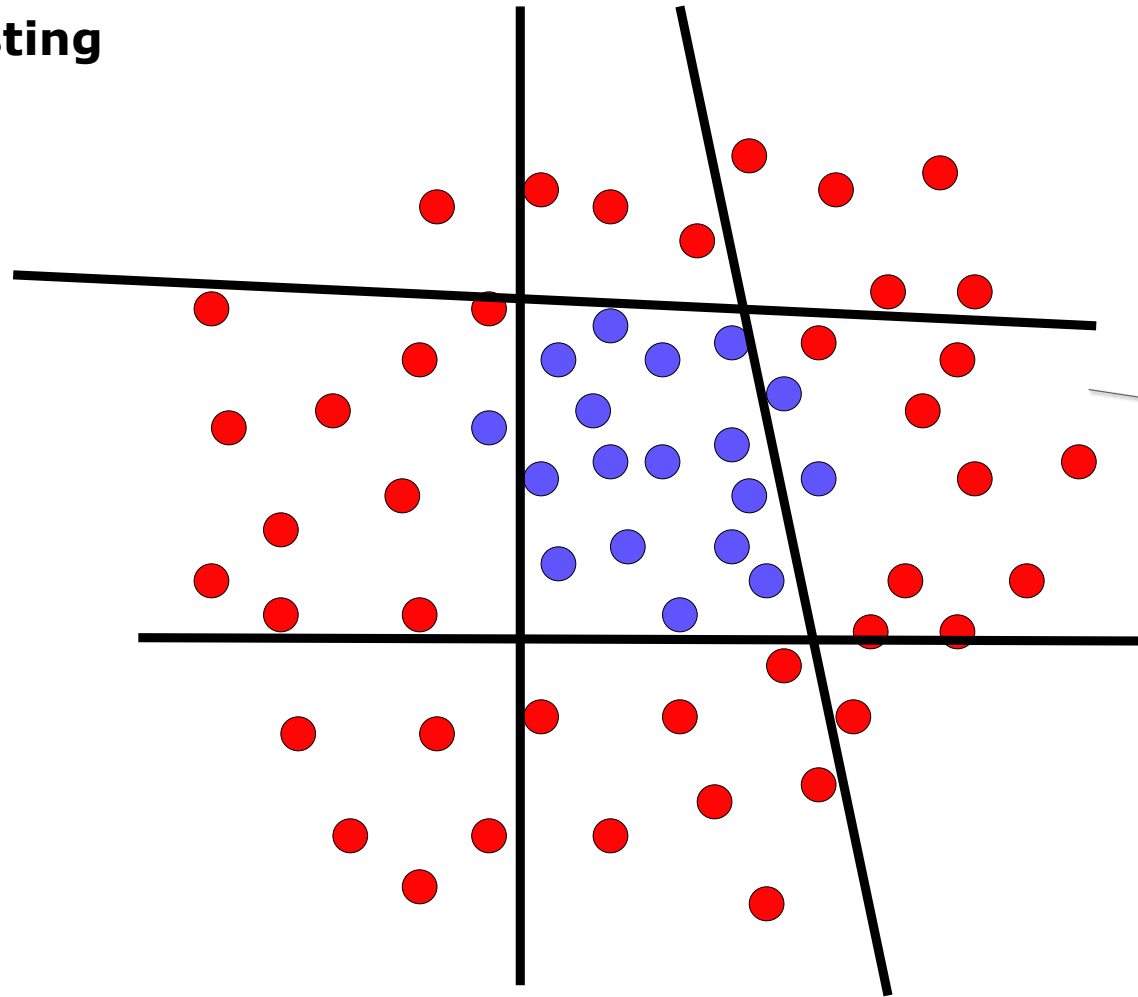
Boosting



Boosting



Boosting



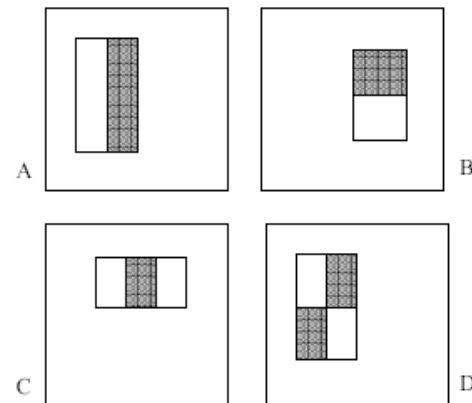
The final strong (non-linear) classifier is built as the combination of all the weak (linear) classifiers

Features

Filter responses (convolutions) can be used as features but are computationally expensive

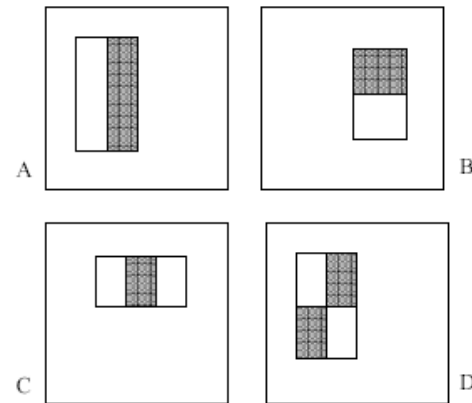
Idea:

- Use simple filters of different shapes and sizes with only -1 or +1 in the filter masks
- These are called *Haar Features*



Feature Extraction

- Calculate features on sub windows (sliding window)
- Sub-Image 24x24 Pixel
- All possible combination of filters are feature candidates: 160'000 combinations



Integral image

Fast calculation of filter results by using integral images

Each value in the integral image contains the sum of all pixel value in the upper, left rectangle
also called summed area tables...

Integral Image

Original Image

0	1	3	5	2	7	10	7	10	9
3	7	6	9	3	8	1	8	5	8
1	11	0	7	13	2	14	2	13	1
14	3	2	7	1	0	9	7	2	12
1	5	15	3	6	6	5	1	10	6
8	1	2	6	7	3	2	11	0	15
7	7	6	0	9	5	10	3	8	1
12	5	6	10	11	3	6	7	9	1

Original Image

0	1	3	5	2	7	10	7	10	9
3	7	6	9	3	8	1	8	5	8
1	11	0	7	13	2	14	2	13	1
14	3	2	7	1	0	9	7	2	12
1	5	15	3	6	6	5	1	10	6
8	1	2	6	7	3	2	11	0	15
7	7	6	0	9	5	10	3	8	1
12	5	6	10	11	3	6	7	9	1

Integral Image

		51							

<https://levelup.gitconnected.com/the-integral-image-4df3df5dce35>

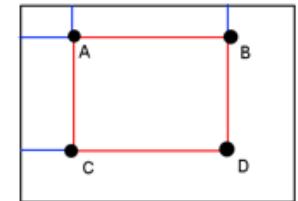
Integral Image for calculating sums

Original Image

0	1	3	5	2	7	10	7	10	9
3	7	6	9	3	8	1	8	5	8
1	11	0	7	13	2	14	2	13	1
14	3	2	7	1	0	9	7	2	12
1	5	15	3	6	6	5	1	10	6
8	1	2	6	7	3	2	11	0	15
7	7	6	0	9	5	10	3	8	1
12	5	6	10	11	3	6	7	9	1

Integral Image

0	1	4	9	11	18	28	35	45	54
3	11	20	34	39	54	65	80	95	112
4	23	32	53	71	88	113	130	158	176
18	40	51	79	98	115	149	173	203	233
19	46	72	103	128	151	190	215	255	291
27	55	83	120	152	178	219	255	295	346
34	69	103	140	181	212	263	302	350	402
46	86	126	173	225	259	316	362	419	472



$$\text{Sum} = D - B - C + A = 215 - 72 - 80 + 20 = 83$$

Classification

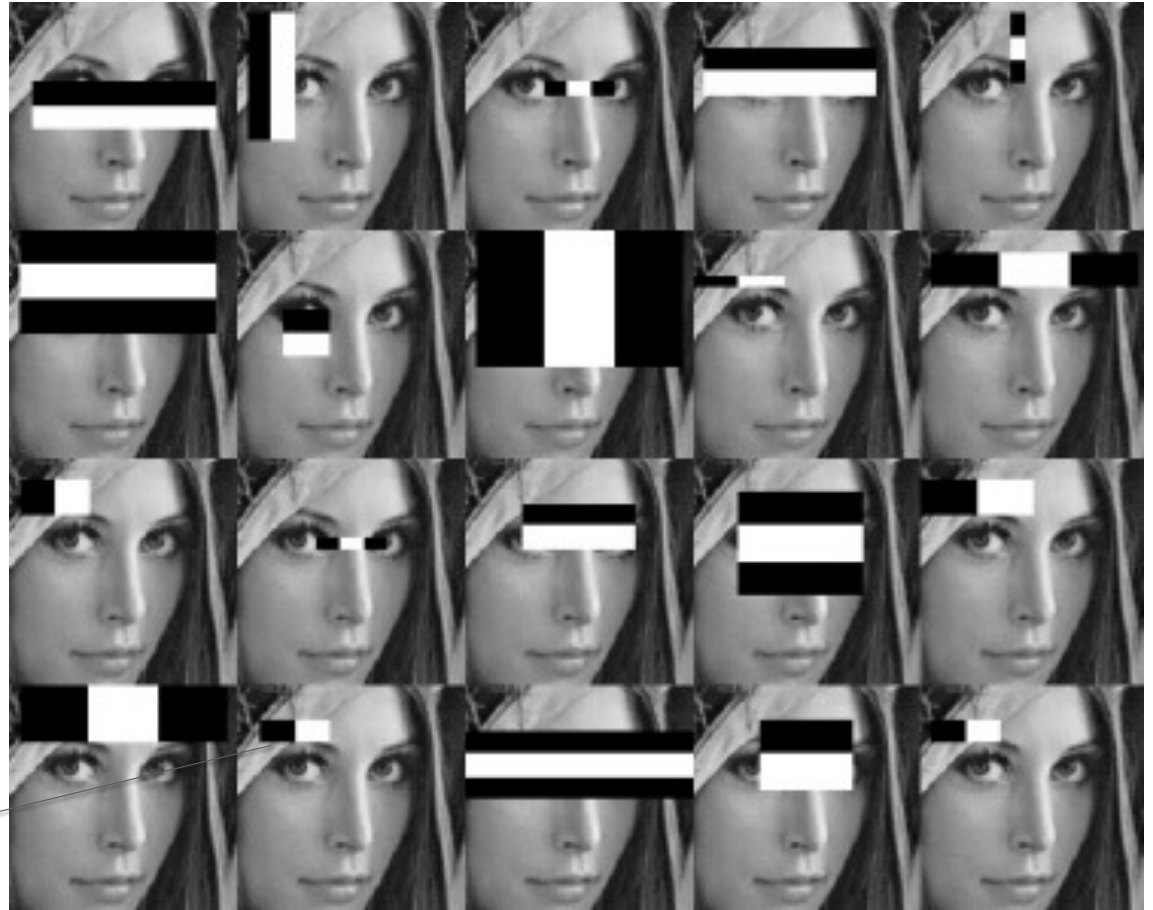
Problem:

- How to evaluate 160'000 features per sub window?

Idea:

- Treat features as weak classifiers
- Train important features using adaboost

Each filter response is a classifier for a face

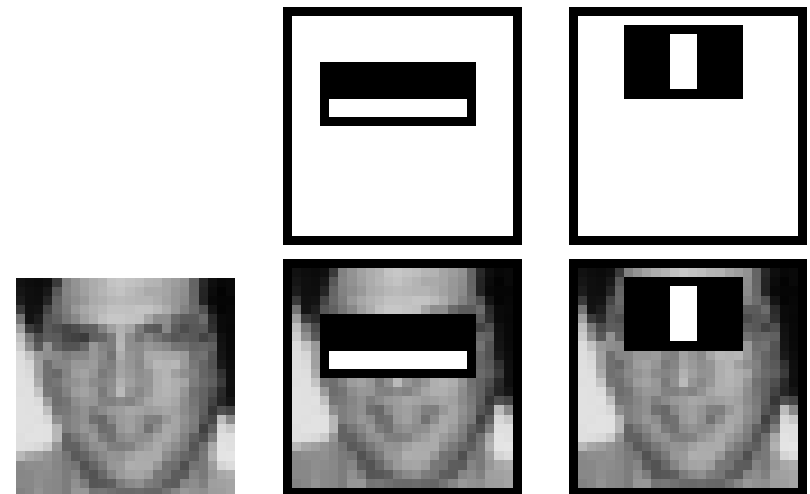


Feature selection by Adaboost

The first and second features selected by Adaboost.

The first feature measures the difference in intensity between the eye region and the face below.

The second feature compares the intensities in the eye region to the intensity across the bridge of the nose.



Efficient Face Detection

A sliding window detector needs to evaluate thousand of possible position/scae combinations in the image

Faces are rare: 10-12 faces per image

Using 1'000'000 pixels should result in a false positive rate of less than 10^{-6}

Idea:

- Dismiss regions without faces efficiently

Goal for face detection:

- **high true positive rate** (85-95%)
- **Low false positive rate** (10^{-5} bis 10^{-6})

Cascade approach:

- Use a sequence of individual classifiers:
- False positive rates multiply
- True positive rates multiply

Metrics: Precision, Recall and more

$$\text{Precision} = \frac{tp}{tp+fp}$$

$$\text{Recall} = \frac{tp}{tp+fn}$$

$$\text{Specificity} = \frac{tn}{tn+fp}$$

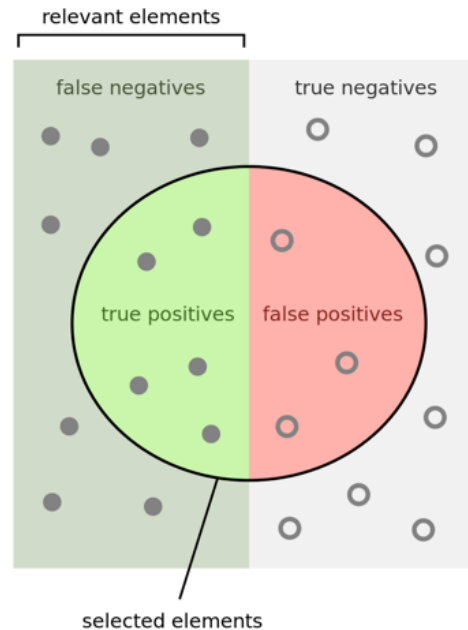
$$\text{Accuracy} = \frac{tp+tn}{tp+tn+fp+fn}$$

F1 = harmonic mean of precision and recall

$$= \frac{2tp}{2tp+fp+fn}$$

Intersection over union (IoU)

$$= \frac{tp}{tp+fp+fn}$$



Confusion Matrix

		Prediction	
		pos	neg
Actual	pos	TP	FN
	neg	FP	TN

How many selected items are relevant?

Precision = $\frac{\text{green}}{\text{green} + \text{red}}$

How many relevant items are selected?

Recall = $\frac{\text{green}}{\text{green} + \text{gray}}$

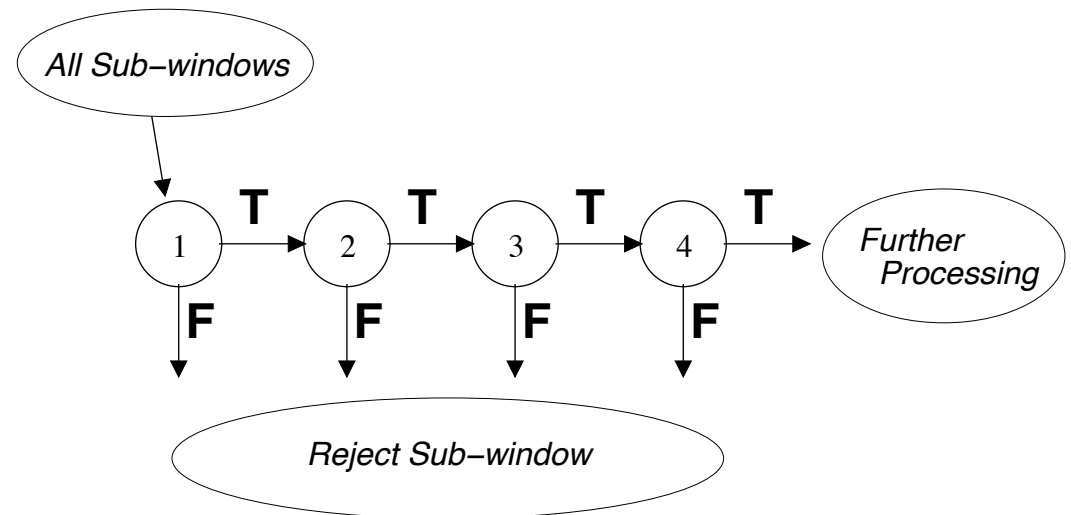
Cascaded Classifiers Approach

Reject false results early in the processing.

With the overall goal, we can calculate the necessary goal for each step of the cascade:

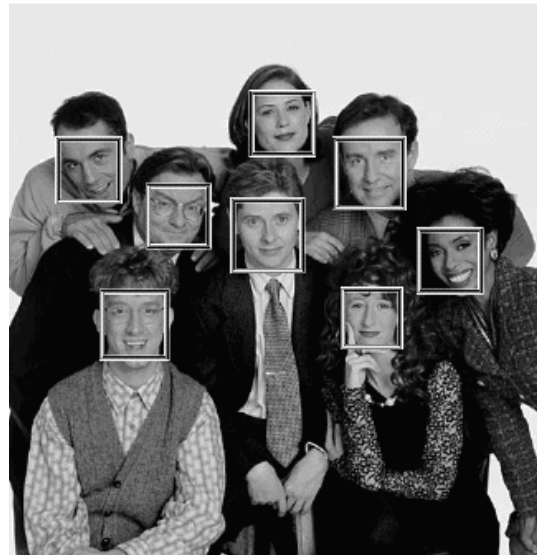
- 0.99 True positive rate
- 0.3 False positive rate

Each step of the 10-step cascade is trained with adaboost using 20 (different) features

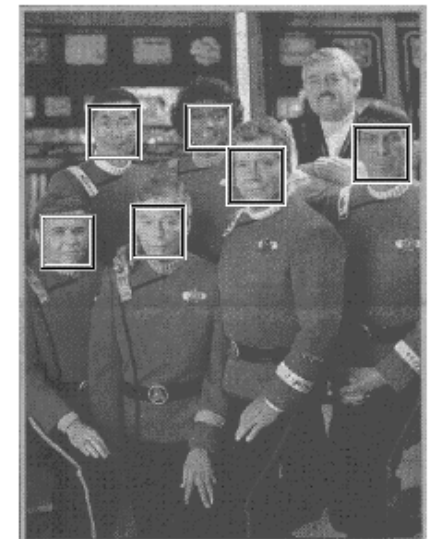




Training data



Results



Example: 22 stage cascade



Other classification approaches

Viola-Jones approach using **Haar-Wavelets** as Features and **Boosting** as Classifier, together with the Cascade Approach is/was a popular object detection and classification approach

Popular other (later) approaches combine other feature detection with SVMs

Pedestrian Detection

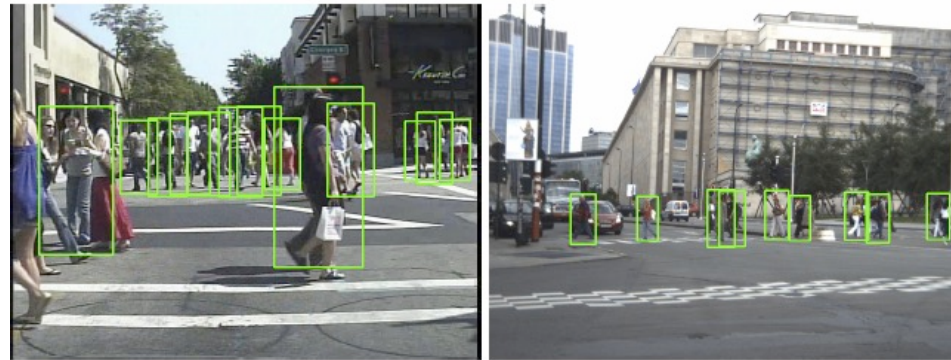
Histograms of Oriented Gradients for Human Detection

Navneet Dalal and Bill Triggs

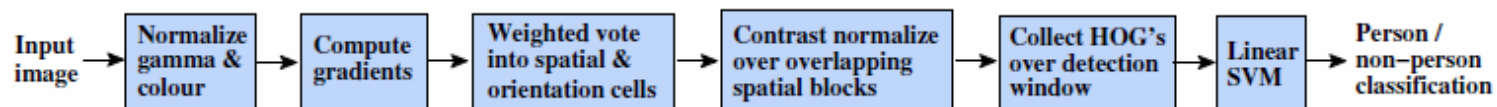
INRIA Rhône-Alpes, 655 avenue de l'Europe, Montbonnot 38334, France
{Navneet.Dalal,Bill.Triggs}@inrialpes.fr, <http://lear.inrialpes.fr>

Use Histogram of Oriented Gradients (HOG) as Features for pedestrian detectors:

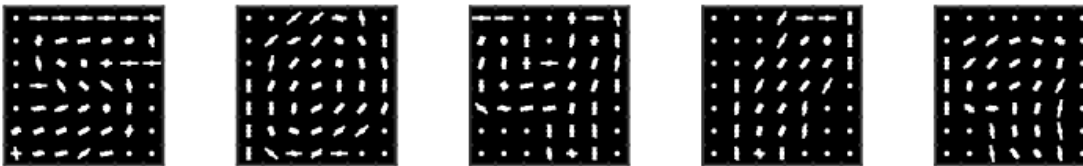
- Calculate HOG Descriptors (for whole image in blocks)
- Collect HOG's in Detection Window
- Use Linear SVM for Person / Non-Person Classification



S. Walk, N. Majer, K. Schindler and B. Schiele, "New features and insights for pedestrian detection," CVPR 2010.



Histogram of Oriented Gradients (HOG)



Zyarah and Kudithipudi (2017) IJCNN

1. Split image into square tiles
2. Compute gradient orientations for each tile



...



3. Compute histogram of gradient orientations

The HOG is a quantitative fingerprint of the object shape.

Problem: The HOG changes when the image is rotated.

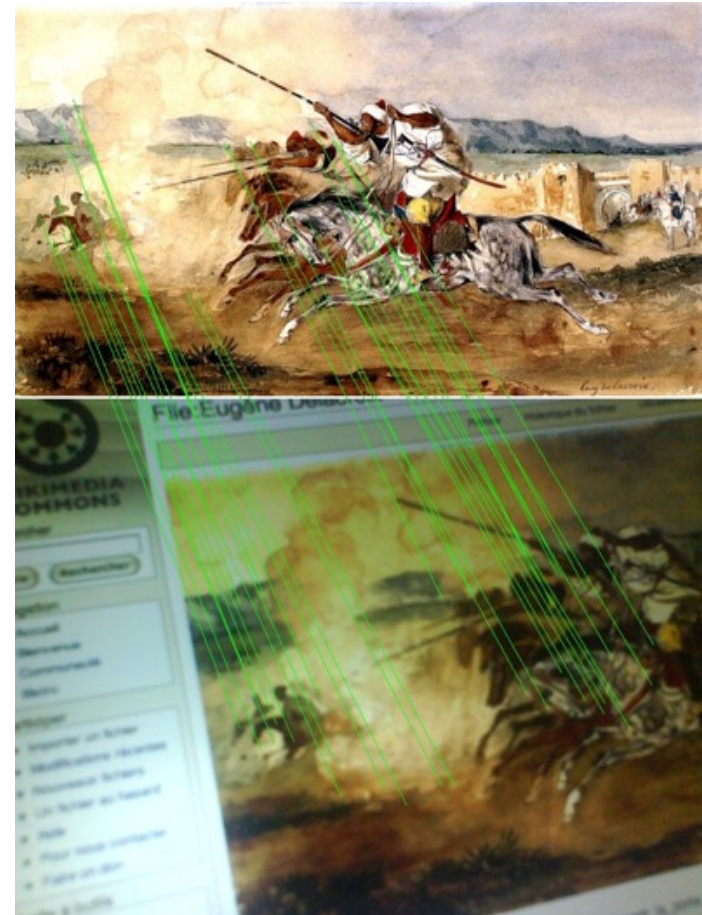
Scale-invariant Feature Transform (SIFT)

- Features calculate properties of an image or an image region
- Besides classification, they are also used to find subregions or for image stitching etc.



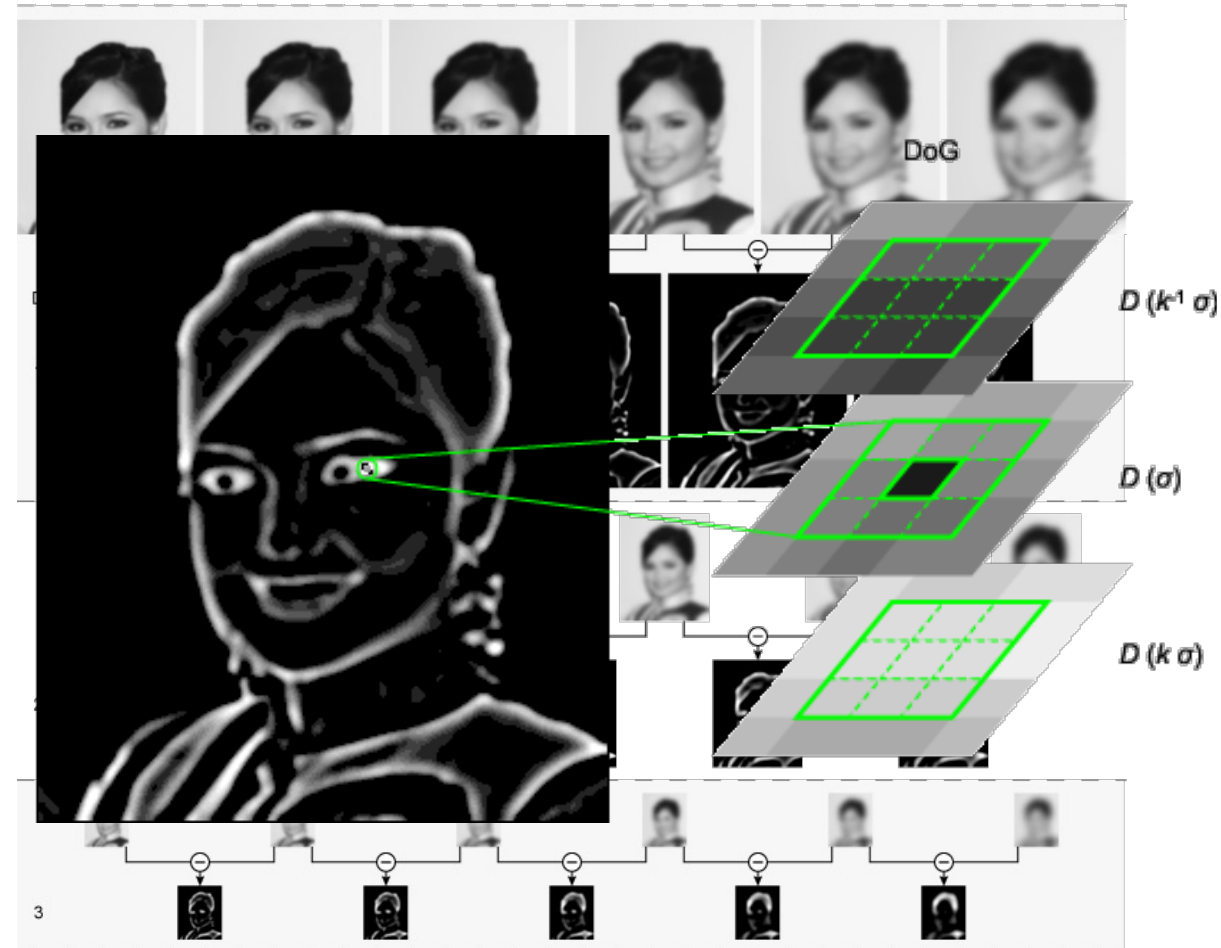
https://fr.wikipedia.org/wiki/Scale-invariant_feature_transform

HSLU 20. März 2025



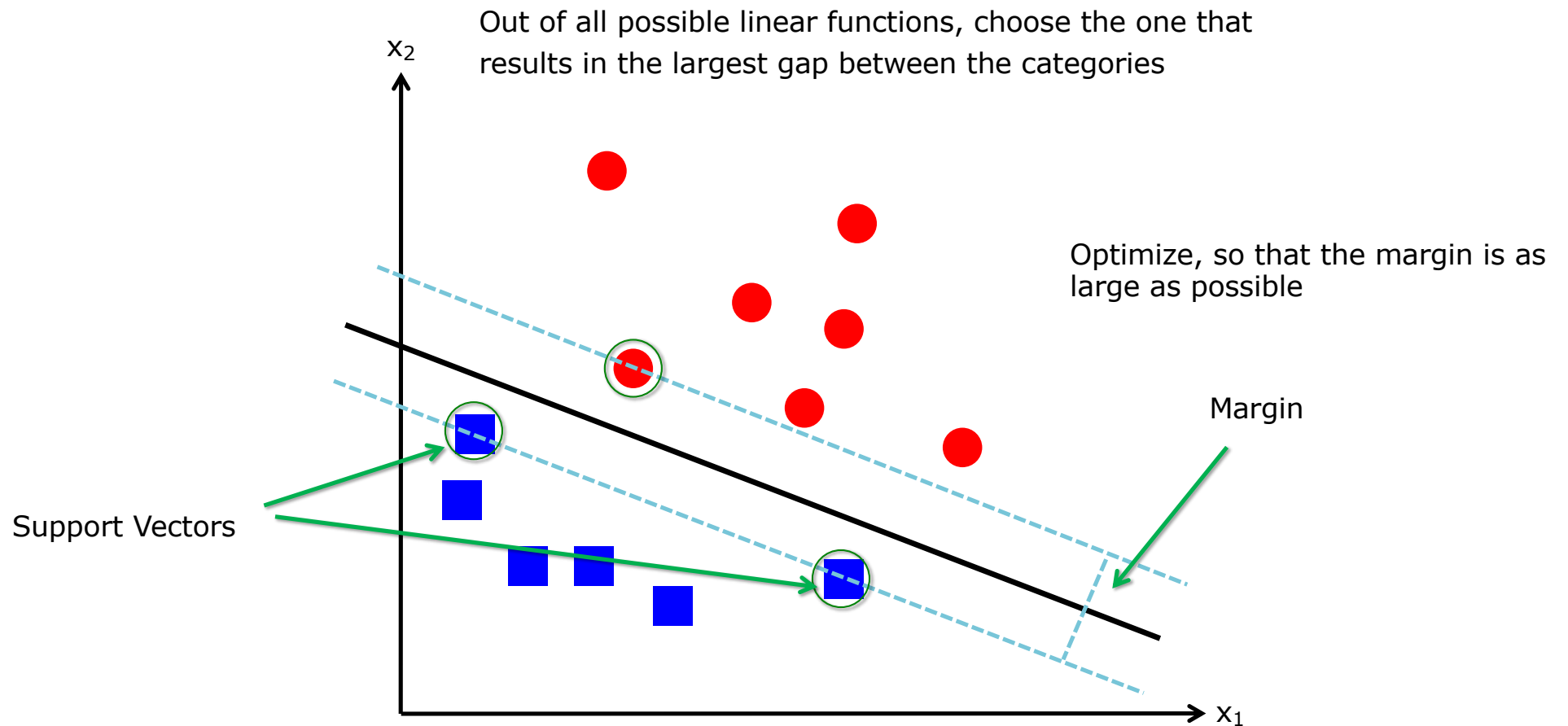
SIFT

- Scale Invariant Feature Transform
- Popular feature descriptor for various tasks such as object matching, image stitching etc.
- Combination of detector + descriptor
- Descriptor uses histogram of gradient
 - **Important: Orientation of gradients is taken relative to orientation of key point**
 - The keypoint descriptor is scale- and orientation-invariant
- Results in a 128 dimensional vector

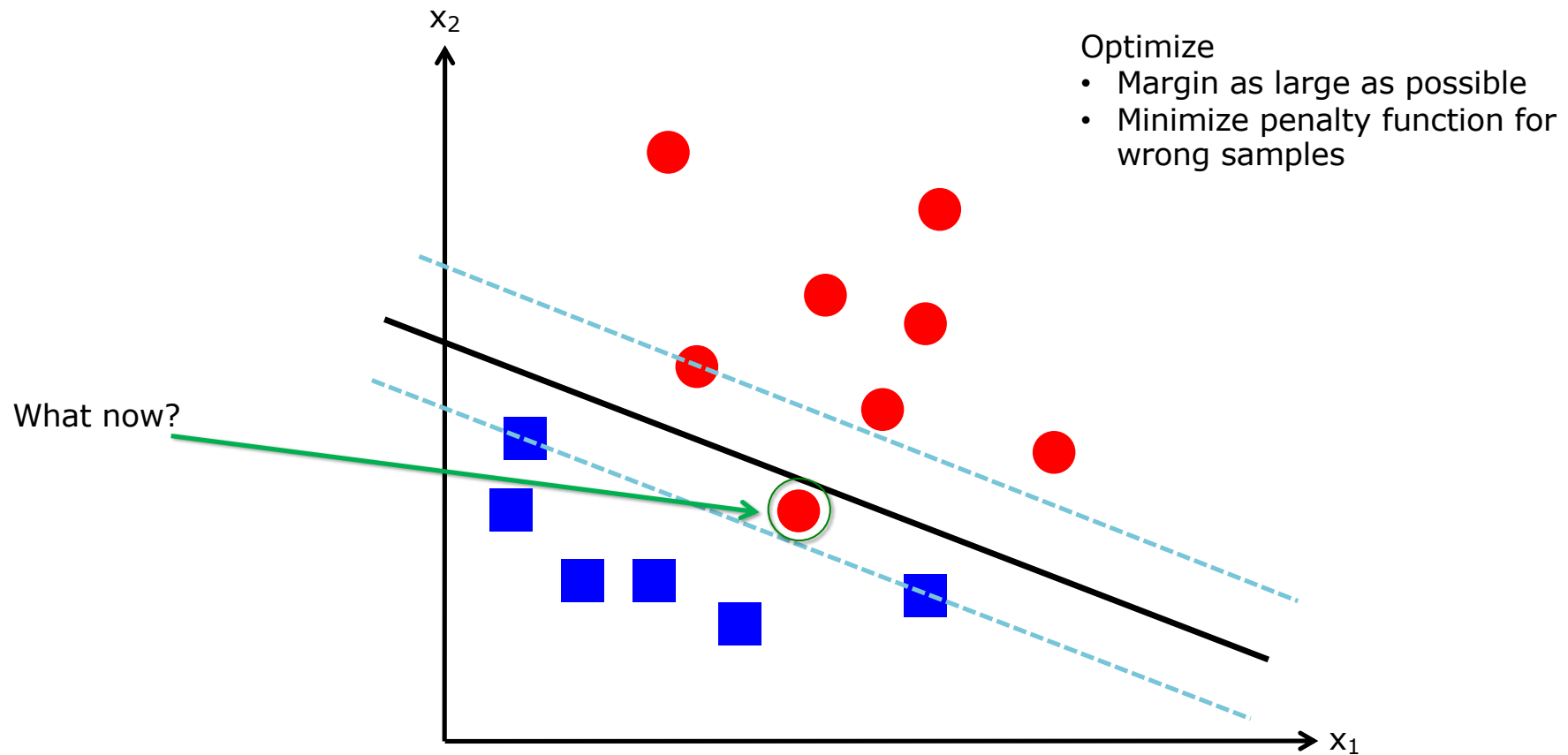


https://fr.wikipedia.org/wiki/Scale-invariant_feature_transform

Support Vector Machine (SVM)

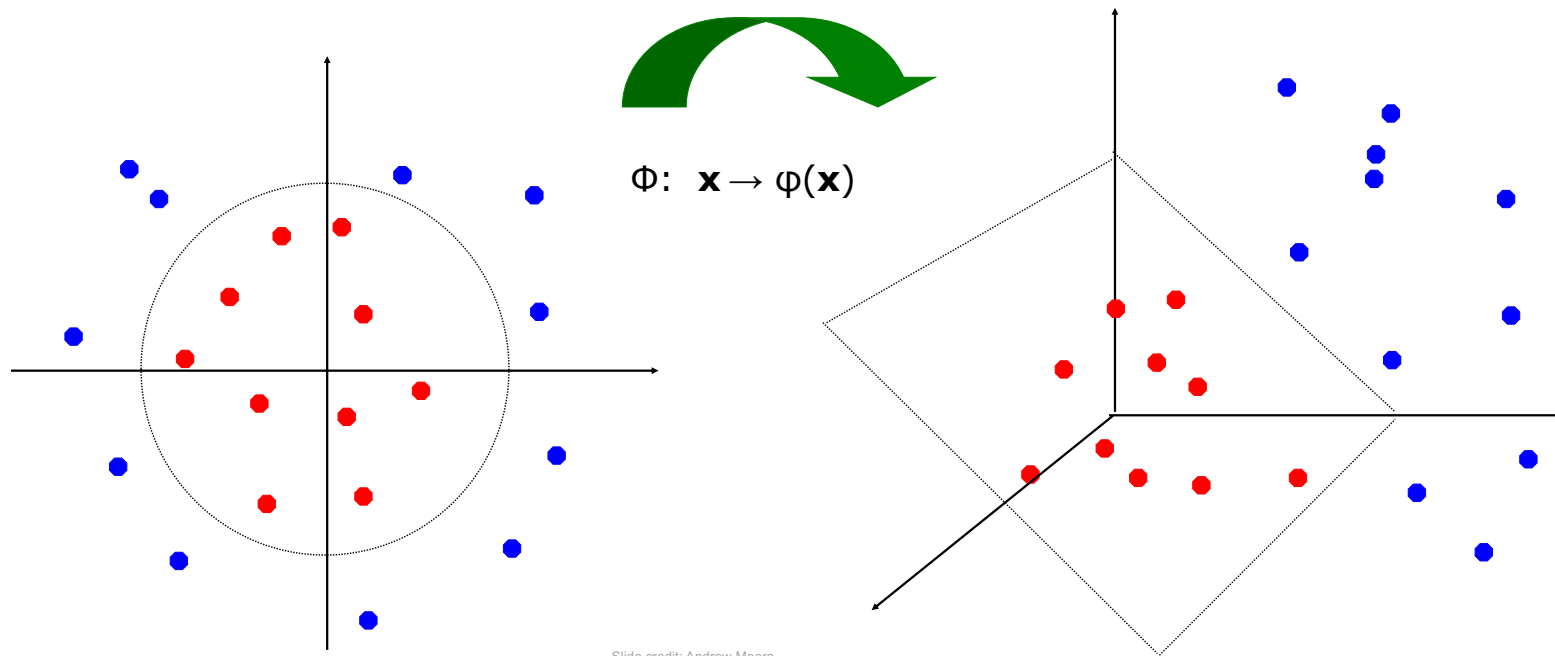


Support Vector Machine

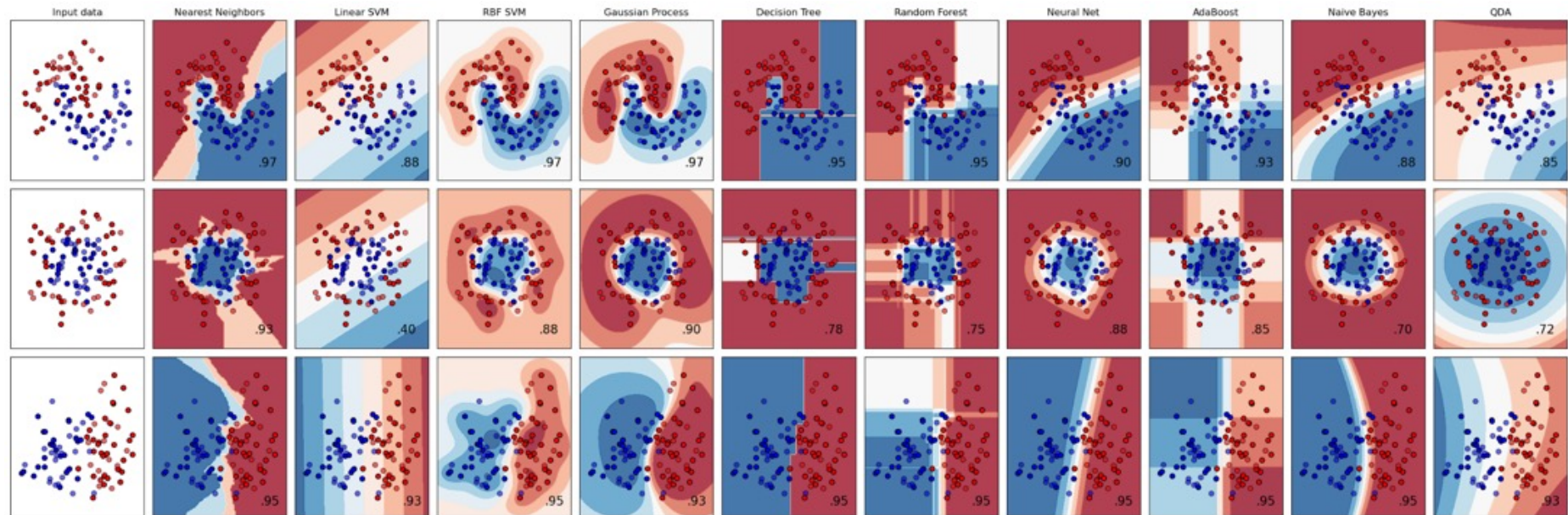


Nonlinear Support Vector Machine

Idea: Transform into higher dimensional space, where the data is linearly separable (kernel trick)



Scikit-learn: machine learning in Python



https://scikit-learn.org/stable/supervised_learning.html

Exercise: Classification on the CIFAR-10 Dataset

Use HOG and SVM for classification

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck

